



Linguagem de Programação: C++

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

Capítulo 1

Fundamentos da Linguagem C++

Resumo

Este capítulo apresenta os fundamentos da programação em C++, estabelecendo uma base conceitual e prática para o desenvolvimento de programas segundo as regras da linguagem e as boas práticas de engenharia de software. Inicialmente, são analisados a estrutura básica de um programa, a função `main`, o processo de compilação e execução e o papel das diretivas de pré-processamento, com ênfase na diretiva `#include` e na utilização de cabeçalhos da Biblioteca Padrão. Em seguida, são introduzidos o espaço de nomes `std`, os mecanismos de entrada e saída por fluxos, incluindo `std::cout`, `std::cin` e `std::getline`, e os conceitos de declaração, definição, inicialização e atribuição de variáveis. O capítulo também aborda a dedução automática de tipos com `auto`, os operadores aritméticos, as regras de precedência e associatividade e os operadores de atribuição composta. Complementarmente, são discutidos os mecanismos de comentários e princípios de documentação e legibilidade do código. Ao longo da exposição, são enfatizadas distinções terminológicas e conceituais fundamentais da linguagem, com o objetivo de evitar ambiguidades entre conceitos como identificadores, palavras-chave, objetos, expressões, instruções, declarações e definições. Exemplos executáveis e orientações de boas práticas são utilizados para relacionar os conceitos formais à prática de desenvolvimento. Ao final, uma revisão conceitual sistematiza os principais termos e distinções apresentados, proporcionando uma referência rápida para os capítulos subsequentes.

Palavras-chave: C++; programação; linguagem de programação; fundamentos de programação; Biblioteca Padrão; espaços de nomes; variáveis; expressões; entrada e saída; boas práticas de programação.

“A simplicidade é a sofisticação suprema.”

— Leonardo da Vinci

1.1 Estrutura Básica de um Programa em C++

Um programa escrito em **C++** é constituído por elementos da linguagem que, combinados segundo as regras sintáticas e semânticas definidas pelo padrão, descrevem um comportamento computacional [1]. Entre esses elementos encontram-se declarações, definições, funções, instruções, expressões e diretivas de pré-processamento. A compreensão dessas categorias é fundamental para interpretar corretamente a estrutura de um programa, bem como, analisar conceitos mais avançados da linguagem.

C++ é uma linguagem de programação de propósito geral, cuja especificação é estabelecida pela norma internacional ISO/IEC 14882 [1]. Obras introdutórias como a de Deitel e Deitel apresentam esses conceitos de forma progressiva, utilizando programas completos para relacionar a sintaxe da linguagem ao comportamento observado durante sua execução [2]. Essa abordagem é particularmente adequada para os primeiros contatos com C++, pois permite analisar cada construção no contexto de um programa funcional.

Nesta seção, será apresentada a estrutura de um programa C++ simples, identificando o papel desempenhado por seus principais componentes. O objetivo é estabelecer uma base conceitual para a compreensão dos programas que serão desenvolvidos nas seções e capítulos seguintes.

O código a seguir apresenta um programa simples que exibe uma mensagem na saída padrão. O exemplo contém elementos fundamentais da estrutura de um programa C++, incluindo uma diretiva de pré-processamento, uma definição da função `main`, uma expressão utilizada para produzir uma saída e uma instrução de retorno [1, 3].

Código 1.1: code_01.cpp

```
1 #include <iostream> // Disponibiliza os recursos de entrada e saída
2
3 // Funcao principal do programa
4 int main()
5 {
6     std::cout << "Bem-vindo ao C++!\n"; // Exibe uma mensagem
7
8     return 0; // Indica termino normal do programa
9 }
```

O exemplo será analisado nas subseções seguintes. A análise não se limitará à função de cada linha, mas também distinguirá os diferentes conceitos sintáticos envolvidos, como diretiva de pré-processamento, definição de função, bloco, expressão, instrução e literal. Essa distinção terminológica será mantida ao longo do capítulo para evitar que diferentes construções da linguagem sejam tratadas genericamente como “instruções”.

1.1.1 Anatomia dos Componentes da Estrutura Básica

A análise detalhada do Código (1.1), permite identificar diferentes elementos sintáticos e semânticos da linguagem C++. Embora o exemplo seja pequeno, ele reúne conceitos fundamentais que serão utilizados ao longo de todo o estudo da linguagem.

Comentários

As linhas iniciadas por duas barras inclinadas (//) constituem *comentários de linha*. Um comentário iniciado por // estende-se até o final da linha física correspondente.

Os comentários são destinados à documentação e à comunicação entre os desenvolvedores. Eles não fazem parte da semântica do programa e não produzem uma operação executável durante a execução da aplicação. O bloco a seguir demonstra sua aplicação.

```
1 int main() {  
2     // Este comentario sera ignorado pelo compilador.  
3     return 0;  
4 }
```

Além dos comentários de linha, C++ também possui comentários delimitados por /* e */, que podem ocupar uma ou várias linhas.

Diretiva de Pré-processamento

A construção:

```
1 #include <iostream>
```

é uma **diretiva de pré-processamento**. Ela não constitui uma instrução C++ convencional. A diretiva `#include` solicita ao pré-processador que processe o conteúdo do arquivo de cabeçalho indicado antes da compilação das unidades de tradução.

Nesse caso, `<iostream>` é um cabeçalho da biblioteca padrão de C++, associado às operações de entrada e saída (I/O) baseadas em fluxos (*streams*). Entre as entidades disponibilizadas por esse cabeçalho encontra-se `std::cout`, utilizado posteriormente no programa.

É importante distinguir, portanto:

- `#include`: diretiva de pré-processamento;
- `<iostream>`: cabeçalho da biblioteca padrão;
- `std::cout`: objeto de fluxo utilizado para saída;
- operador `'<<'`: utilizado para inserir dados no fluxo.

Essa distinção será importante posteriormente, quando estudarmos o processo de compilação, a organização de programas em múltiplos arquivos e a utilização de bibliotecas.

A Função Principal

A construção:

```
1 int main() {  
2     return 0;  
3 }
```

define uma função denominada `main`. Em um programa C++ destinado a execução, a função `main` constitui o ponto de entrada da execução do programa. O especificador de tipo `int`, colocado antes do nome da função, indica que a função possui tipo de retorno inteiro. A lista vazia `()` indica, nessa forma particular, que a função não recebe parâmetros.

É importante utilizar aqui a terminologia correta: nesse contexto, `main` não está apenas sendo declarada; o código apresentado **define** a função, pois fornece também o seu corpo.

Para os exemplos introdutórios deste capítulo, podemos considerar `main` como a função a partir da qual começa a execução de um programa C++. Existem, entretanto, diferentes formas válidas de definição de `main`, incluindo formas que recebem argumentos da linha de comando. Esse assunto será retomado posteriormente.

Chaves e Bloco de Código

As chaves:

```
1 {  
2     // corpo do bloco  
3 }
```

delimitam um **bloco**. No caso da função `main`, as chaves delimitam o corpo da função.

O bloco não deve ser confundido com a própria função. A função é uma entidade que possui, entre outras propriedades, um nome, parâmetros e um corpo; o bloco corresponde à sequência de declarações e instruções delimitada pelas chaves.

Além disso, blocos estão relacionados às regras de **escopo** da linguagem. Uma variável declarada localmente em um bloco, por exemplo, pode ter seu uso limitado ao escopo correspondente.

Essa distinção entre *função*, *bloco* e *escopo* será particularmente importante quando forem estudadas variáveis e estruturas de controle.

A Operação de Saída com `std::cout`

A expressão:

```
1 std::cout << "Bem-vindo ao C++!\n";
```

é utilizada para enviar dados ao fluxo de saída padrão.

O identificador qualificado `std::cout` refere-se ao objeto de fluxo de saída padrão definido pela biblioteca padrão. O prefixo `std::` indica que a entidade pertence ao *namespace* `std`.

O operador '«', nesse contexto, é utilizado para inserir dados no fluxo. A cadeia de caracteres "Bem-vindo ao C++!\n" é um literal de string. O trecho '\n' representa um caractere de nova linha dentro da sequência de caracteres.

A expressão completa é seguida por um ponto e vírgula:

```
1 std::cout << "Bem-vindo ao C++!\n";
```

O ponto e vírgula desempenha, nesse caso, o papel de terminador da instrução de expressão.

É importante observar que `std::cout`, isoladamente, não é uma "instrução". A instrução é formada pela expressão de saída seguida pelo ponto e vírgula.

A Instrução de Retorno

A construção:

```
1 return 0;
```

é uma **instrução de retorno**. Ela encerra a execução da função corrente e, no caso de `main`, fornece um valor de retorno ao ambiente que iniciou a execução do programa. Como `main` possui tipo de retorno `int`, o valor 0 é compatível com o tipo de retorno da função.

No contexto de um programa executável, o retorno de 0 por `main` é utilizado convencionalmente para indicar término normal da execução. Valores diferentes de zero são frequentemente utilizados para indicar alguma condição de erro ou situação excepcional, embora o significado específico desses valores dependa do programa e do ambiente de execução.

Existe ainda uma particularidade importante da linguagem: ao atingir a chave de fechamento da função `main` sem executar explicitamente uma instrução `return`, o programa equivale, nesse ponto, ao retorno do valor 0. Portanto, o exemplo também poderia ser escrito como:

```
1 int main() {  
2     // Execução do programa.  
3 }
```


Essa regra é específica de `main` e não deve ser generalizada para todas as funções que possuem tipo de retorno diferente de `void`.

1.1.2 Compilação e Execução do Programa

O código-fonte de um programa C++ não é, em geral, executado diretamente pelo sistema operacional. Antes de sua execução, o código precisa passar por um processo de tradução e construção que transforma as construções escritas pelo programador em uma forma que possa ser executada pelo ambiente computacional [1, 4].

Em uma visão simplificada, o processo de construção de um programa C++ pode ser representado pelas seguintes etapas:

1. **Pré-processamento:** diretivas como `#include` são processadas, produzindo uma unidade de tradução;
2. **Compilação:** a unidade de tradução é analisada de acordo com as regras da linguagem e transformada em código apropriado para a plataforma de destino;
3. **Ligação (linking):** os arquivos objeto e as bibliotecas necessárias são combinados para formar, quando apropriado, um programa executável;
4. **Execução:** o sistema operacional carrega e inicia o programa construído.

Essa descrição representa uma simplificação do processo real. Compiladores modernos podem realizar etapas adicionais, como otimizações, geração de diferentes formas de código intermediário e outras transformações dependentes da implementação. Para os objetivos introdutórios deste capítulo, entretanto, é suficiente compreender que o comando utilizado pelo programador para “compilar” um programa geralmente coordena várias dessas etapas.

No exemplo apresentado anteriormente, o arquivo-fonte `code_01.cpp` (Código (1.1)) pode ser construído utilizando um compilador C++ disponível no sistema. A seguir são apresentados exemplos de comandos para diferentes sistemas operacionais.

Windows

No Windows, uma possibilidade é utilizar o compilador GCC por meio de ambientes como MinGW ou MSYS2. Considerando que o compilador g++ esteja instalado e disponível no PATH, o programa pode ser construído a partir do Prompt de Comando ou do PowerShell:

1. Compilação:

```
g++ code_01.cpp -o code_01.exe
```

A opção `-o` especifica o nome do arquivo de saída. Nesse exemplo, o arquivo produzido será denominado `code_01.exe`.

2. Execução:

```
.\code_01.exe
```

O comando de execução apresentado utiliza a sintaxe característica do PowerShell. Dependendo do ambiente utilizado, a forma de executar o programa pode variar.

Linux

Em sistemas Linux, considerando que o compilador GCC esteja instalado, o programa pode ser construído utilizando o comando g++:

1. Compilação:

```
g++ code_01.cpp -o code_01
```

2. Execução:

```
./code_01
```

O prefixo `./` especifica o caminho relativo correspondente ao diretório atual. Dessa forma, o sistema é instruído a procurar o executável `code_01` nesse diretório.

macOS

No macOS, é comum utilizar o conjunto de ferramentas de desenvolvimento baseado em Clang/LLVM. Considerando que as ferramentas de linha de comando estejam instaladas, o programa pode ser construído com `clang++`:

1. Compilação:

```
clang++ code_01.cpp -o code_01
```

2. Execução:

```
./code_01
```

O comando `clang++` é utilizado especificamente para a compilação de programas C++. Embora algumas instalações disponibilizem comandos compatíveis ou aliases adicionais, recomenda-se utilizar `clang++` explicitamente quando o objetivo é deixar claro que o programa está sendo construído como código C++.

Observação sobre o processo de construção

Os comandos apresentados anteriormente utilizam uma única invocação do compilador para simplificar o processo para o iniciante. Internamente, entretanto, a construção de um programa pode envolver diversas etapas e arquivos intermediários.

Por exemplo, em um projeto composto por vários arquivos-fonte, cada arquivo `.cpp` pode ser compilado separadamente para produzir um arquivo objeto. Posteriormente, esses arquivos podem ser ligados entre si e às bibliotecas necessárias para produzir o executável final.

Essa separação entre **código-fonte**, **compilação** e **ligação** será particularmente importante quando forem estudados programas compostos por múltiplos arquivos.

👍 Boa Prática de Programação 1.1

Utilize um processo de construção reprodutível e mantenha explícito, sempre que possível, o compilador e as opções utilizadas para construir o programa. Em projetos maiores, recomenda-se utilizar um sistema de build, como CMake, em vez de depender exclusivamente de comandos digitados manualmente no terminal.

👍 Boa Prática de Programação 1.2

Mantenha uma convenção de indentação consistente em todo o projeto. A escolha entre dois, quatro ou outro número de espaços é secundária em relação à consistência da convenção adotada e às diretrizes de estilo utilizadas no projeto.

⚠ Erro Comum de Programação 1.1: Confundir compilação com execução

Compilar um programa e executar um programa são operações distintas. Um comando como `g++ code_01.cpp -o code_01` constrói o programa; ele não executa automaticamente o arquivo resultante. A execução deve ser realizada posteriormente utilizando o executável produzido.

1.2 A Função `main` e seu Papel como Ponto de Entrada

Em C++, a função `main` possui um papel especial: ela constitui o ponto de entrada designado pela linguagem para a execução do programa [1, 2, 4]. É importante, entretanto, distinguir o conceito de *ponto de entrada* da implementação concreta do processo de inicialização de um programa.

O sistema operacional não precisa, necessariamente, localizar diretamente a função `main` para iniciar sua execução. O executável pode conter código de inicialização fornecido pelo ambiente de execução, que realiza operações necessárias à preparação do programa e, posteriormente, conduz a execução até a função `main`. Os detalhes desse mecanismo dependem da plataforma, do compilador e do ambiente de execução.

Assim, para fins introdutórios, podemos representar o processo de forma simplificada como:

carregamento do programa → *inicialização do ambiente* → `main` → *execução do programa*

Essa distinção permite compreender corretamente o papel de `main` sem associá-lo indevidamente a detalhes específicos de um determinado sistema operacional ou compilador.

1.2.1 Assinatura e Anatomia da Função `main`

Uma das formas mais simples e conhecidas de definição da função `main` é através da sua assinatura básica, conforme o código a seguir:

```
1 int main()
2 {
3     // Corpo da funcao
4     return 0;
5 }
```

Essa construção pode ser analisada em diferentes componentes:

- **Tipo de retorno (`int`):** especifica que a função possui tipo de retorno inteiro. No caso de `main`, esse valor participa do mecanismo pelo qual o término da execução do programa é comunicado ao ambiente hospedeiro [1].
- **Nome da função (`main`):** `main` é uma função com significado especial definido pela linguagem C++. Seu papel como ponto de entrada não decorre apenas de ser um identificador com determinado nome, mas de uma regra específica da linguagem [1].
- **Lista de parâmetros (`()`):** nessa forma de definição, a função `main` não possui parâmetros. Existem outras formas válidas de definição de `main`, incluindo aquelas que permitem ao programa receber argumentos fornecidos no momento de sua inicialização [1, 3].
- **Bloco da função (`{ }`):** as chaves delimitam o bloco que constitui o corpo da função. As declarações e instruções contidas nesse bloco fazem parte da

definição da função. O bloco também está relacionado às regras de escopo da linguagem, mas não deve ser confundido com o próprio conceito de escopo.

Posteriormente, serão apresentadas as formas que permitem receber argumentos da linha de comando e outras características relacionadas à função principal.

1.2.2 O Mecanismo do Valor de Retorno e `return 0;`

A instrução `return 0;` encerra a execução da função `main` retornando o valor inteiro 0. Como `main` possui um papel especial em C++, o valor retornado é utilizado como *status de término* do programa e é disponibilizado ao ambiente que iniciou sua execução [1, 4].

É importante distinguir, portanto, o **valor de retorno da função** do **status de término do programa**. No caso específico de `main`, a linguagem estabelece uma relação entre esses dois conceitos: quando a função termina por meio de uma instrução `return`, o valor retornado é utilizado no encerramento do programa [1, 3].

De forma geral, convencionou-se utilizar o valor 0 para indicar término bem-sucedido e valores diferentes de zero para representar alguma condição diferente do término normal. Entretanto, os significados específicos dos valores diferentes de zero podem depender do ambiente de execução e da convenção adotada pelo programa ou sistema. Portanto, não é correto afirmar que todo valor diferente de zero representa necessariamente um determinado tipo de erro [1, 3].

Podemos representar a ideia de forma simplificada:

`return 0;` $\longrightarrow$ *término normal de `main`* $\longrightarrow$ *status de término igual a zero*

Para outros valores, o programa pode comunicar ao ambiente uma condição diferente de sucesso:

`return valor;` $\longrightarrow$ *término de `main`* $\longrightarrow$ *status de término correspondente ao valor retornado*

O significado concreto desse status para um processo ou para um programa que o tenha iniciado depende do ambiente de execução. A linguagem C++ define a relação entre o

retorno de `main` e o encerramento do programa, mas não estabelece uma classificação universal para todos os valores diferentes de zero.

O retorno implícito de `main`

Uma característica especial da função `main` é que o programador não precisa escrever explicitamente uma instrução `return` ao final de seu corpo. Caso o fluxo de execução alcance a chave de fechamento da função `main` sem encontrar uma instrução `return`, o efeito é equivalente à execução de:

```
1 int main()
2 {
3     // Instrucoes do programa
4 }
```

Nesse caso, o comportamento é equivalente a:

```
1 int main()
2 {
3     // Instrucoes do programa
4     return 0;
5 }
```

Essa regra é uma propriedade específica da função `main`. Ela não deve ser generalizada para todas as funções que possuem um tipo de retorno diferente de `void`. Nas demais funções que retornam um valor, a ausência de um retorno apropriado pode resultar em comportamento indefinido quando o valor da função é utilizado [1, 5].

Boa Prática de Programação 1.3

Embora o `return 0;` explícito seja opcional ao final de `main`, sua utilização pode ser adotada como convenção de estilo quando se deseja tornar visualmente explícito que a função terminou com status de sucesso. Em código moderno, entretanto, omitir o retorno explícito também constitui uma forma perfeitamente válida e especificada pela linguagem.

1.2.3 Formas Válidas da Função `main`

Embora tenhamos utilizado anteriormente a forma simplificada `int main()`, a linguagem C++ define formas específicas para a função `main` em um ambiente hospedado. Para os objetivos deste capítulo, duas formas fundamentais são:

1. Sem argumentos de linha de comando:

```
1 int main()
2 {
3     // Corpo do programa
4 }
```

Nessa forma, a função `main` não recebe parâmetros. Ela é adequada para programas que não precisam acessar argumentos fornecidos no momento de sua inicialização.

2. Com argumentos de linha de comando:

```
1 int main(int argc, char* argv[])
2 {
3     // Corpo do programa
4 }
```

Nessa forma, os parâmetros `argc` e `argv` permitem que o programa acesse os argumentos de linha de comando disponibilizados para ele durante sua inicialização. O parâmetro `argc` representa a quantidade de argumentos, enquanto `argv` fornece acesso às sequências de caracteres que representam esses argumentos [1, 3].

A utilização de argumentos de linha de comando será aprofundada posteriormente, quando forem estudados parâmetros de funções, arrays e manipulação de strings.

A função `main` possui tipo de retorno `int`. Assim, a seguinte definição não corresponde a uma forma válida de `main` em um programa C++ hospedado conforme o padrão:

```
1 void main()
2 {
3     // ...
4 }
```


O tipo de retorno `void` não é uma alternativa válida ao `int` para a função `main`. Portanto, programas C++ que pretendam seguir o padrão devem utilizar uma das formas válidas de `main` especificadas pela linguagem [1].

❗ Erro Comum de Programação 1.2: Utilizar `void main()`

Não utilize `void main()` em programas C++ que devam ser compatíveis com o padrão da linguagem. A função `main` possui formas específicas definidas pela norma, e o tipo de retorno `int` faz parte dessas formas.

Uma única função `main`

Outro detalhe muito importante é que um programa executável em um ambiente hospedado deve possuir uma única função `main` que desempenha o papel de ponto de entrada do programa. Portanto, não se deve definir múltiplas funções `main` destinadas ao mesmo programa executável.

Em um projeto constituído por vários arquivos-fonte, por exemplo, não é válido definir uma função `main` em cada arquivo e posteriormente tentar construir todos esses arquivos como um único executável. Essa situação normalmente resultará em um erro durante a construção do programa, frequentemente manifestado pelo vinculador (*linker*) como múltiplas definições da função `main`.

❗ Erro Comum de Programação 1.3: Definir múltiplas funções `main`

Não defina mais de uma função `main` para o mesmo programa executável. Em projetos com vários arquivos-fonte, apenas uma função `main` deve desempenhar o papel de ponto de entrada do executável. Outras funções podem ser definidas normalmente e utilizadas pelo programa.

NOTA

Na forma de definição da função `main` que recebe argumentos de linha de comando, os parâmetros `argc` e `argv` permitem ao programa acessar as informações fornecidas no momento de sua inicialização. O parâmetro `argc` é

um valor inteiro que indica a quantidade de argumentos disponibilizados ao programa, enquanto `argv` fornece acesso às sequências de caracteres que representam esses argumentos.

De forma simplificada, considere a definição:

```
1 int main(int argc, char* argv[])
2 {
3     // argc: quantidade de argumentos
4     // argv: argumentos representados como sequencias de caracteres
5     return 0;
6 }
```

Na convenção usual, `argv[0]` contém uma representação do nome ou do caminho utilizado para iniciar o programa, enquanto os argumentos fornecidos explicitamente aparecem a partir de `argv[1]`. Assim, se um programa for executado, por exemplo, como:

```
./programa valor1 valor2
```

teremos, conceitualmente, três elementos acessíveis por meio de `argv`: `argv[0]`, `argv[1]` e `argv[2]`. Consequentemente, nesse exemplo, `argc` terá o valor 3.

Os detalhes de arrays, ponteiros e strings envolvidos na declaração `char* argv[]` serão estudados posteriormente. Neste momento, é suficiente compreender que `argc` informa a quantidade de argumentos e `argv` permite acessar seus respectivos textos.

1.3 Inclusão de Cabeçalhos com `#include`

Programas C++ frequentemente utilizam componentes fornecidos pela Biblioteca Padrão da linguagem ou definidos em outros arquivos do próprio projeto. Para tornar determinadas declarações e definições disponíveis em uma unidade de tradução, o C++ fornece diretivas de pré-processamento, entre as quais se destaca `#include` [1, 2, 4].

É importante distinguir, desde o início, três conceitos relacionados: **arquivo de cabeçalho**, **biblioteca** e **diretiva `#include`**. Um arquivo de cabeçalho é um arquivo que pode conter declarações, definições e outras construções que serão disponibilizadas ao código que o incluir. Uma biblioteca, por outro lado, é um conjunto de componentes que pode envolver código já compilado e outros recursos necessários à construção ou à execução de um programa. A diretiva `#include` é uma instrução destinada ao pré-processamento que permite incorporar o conteúdo processado de determinado arquivo à unidade de tradução [1].

No exemplo do Código (1.1), a diretiva `#include <iostream>` torna disponíveis ao restante da unidade de tradução as declarações fornecidas pelo cabeçalho `<iostream>`. É por meio dessas declarações que o programa pode utilizar componentes de entrada e saída, como `std::cout`.

1.3.1 O Papel do Pré-processador e das Diretivas

O pré-processamento constitui uma etapa conceitual do processo de construção de um programa C++ na qual são processadas diretivas específicas da linguagem, como `#include`, `#define` e as diretivas de compilação condicional. Essas diretivas são iniciadas pelo caractere `#` e possuem regras próprias, distintas das construções da linguagem C++ propriamente dita [1].

Uma diretiva de pré-processamento não é uma instrução C++ convencional. Por esse motivo, uma diretiva como:

```
1 #include <iostream>
```

não recebe um ponto e vírgula ao final.

Em uma aproximação didática, o efeito de `#include` pode ser entendido como se o conteúdo do arquivo especificado fosse incorporado ao local em que a diretiva aparece. Essa analogia é útil para compreender o processo, embora o mecanismo especificado pela linguagem seja mais preciso do que uma simples operação manual de copiar e colar texto [1].

Assim, podemos representar conceitualmente o processo da seguinte forma:

`codigo-fonte` → *pré-processamento* → *unidade de tradução* → *compilação*

A unidade de tradução resultante constitui a entrada para as etapas posteriores de tradução do programa. Essa distinção será importante quando forem estudados arquivos de cabeçalho, múltiplos arquivos-fonte e sistemas de construção.

1.3.2 Sintaxes de Inclusão: Chaves Angulares vs. Aspas Duplas

A diretiva `#include` admite duas formas principais de especificação do arquivo a ser incluído. A diferença está na forma como o nome do arquivo é delimitado e, conseqüentemente, nas regras de busca utilizadas para localizá-lo.

As duas formas são:

- `#include <arquivo>`
- `#include "arquivo"`

A escolha entre essas formas não deve ser entendida simplesmente como uma distinção entre “biblioteca padrão” e “arquivo do usuário”. Embora seja comum utilizar a primeira forma para cabeçalhos fornecidos pelo ambiente de desenvolvimento e a segunda para cabeçalhos do próprio projeto, a diferença fundamental está na estratégia de localização do arquivo especificada pela linguagem e pela implementação do compilador [1, 4].

1. Inclusão com chaves angulares (<...>):

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
```

Nessa forma, o nome do arquivo é delimitado pelos caracteres `<` e `>`. A implementação procura o arquivo nos locais designados para inclusões desse tipo. Esses locais normalmente incluem os diretórios utilizados pelo compilador para os cabeçalhos da Biblioteca Padrão e outros diretórios de inclusão configurados no ambiente de desenvolvimento.

Portanto, não é correto interpretar as chaves angulares como uma indicação de que o arquivo pertence necessariamente à Biblioteca Padrão. Um projeto também pode configurar diretórios adicionais que sejam pesquisados por meio dessa forma.

Aviso

A Biblioteca Padrão do C++ fornece diversos cabeçalhos com nomes como `<iostream>`, `<vector>`, `<string>` e `<cmath>`. Alguns desses cabeçalhos possuem relação direta com componentes provenientes da biblioteca padrão da linguagem C.

Por exemplo, `<cmath>` fornece as funcionalidades matemáticas correspondentes ao cabeçalho tradicional da linguagem C `<math.h>`, mas pertence à interface especificada pela Biblioteca Padrão do C++.

2. Inclusão com aspas duplas ("..."):

```
1 #include "minha_biblioteca.h"
2 #include "modulo/utilitarios.hpp"
```

Nessa forma, o nome do arquivo é delimitado por aspas duplas. A busca considera inicialmente uma localização associada ao arquivo que contém a diretiva, conforme as regras da linguagem e da implementação. Caso o arquivo não seja encontrado, outros diretórios de inclusão podem ser considerados de acordo com a configuração do ambiente de compilação [1].

Essa forma é frequentemente utilizada para incluir cabeçalhos pertencentes ao próprio projeto. Por exemplo, suponha que o projeto possua a seguinte estrutura:

```
projeto/
|-- main.cpp
|-- calculadora.hpp
|-- calculadora.cpp
```

O arquivo `main.cpp` pode incluir o cabeçalho do projeto por meio de:

```
1 #include "calculadora.hpp"
```

Nesse caso, a utilização de aspas duplas expressa a intenção de incluir um arquivo de cabeçalho associado ao próprio projeto, sem que isso constitua, por si só, uma exigência de que o arquivo pertença ao projeto.

NOTA

As formas `#include <arquivo>` e `#include "arquivo"` não devem ser interpretadas como sinônimos estritos de “biblioteca” e “arquivo do usuário”. A diferença está principalmente na forma como a implementação realiza a busca pelo arquivo. Os diretórios efetivamente pesquisados dependem também do compilador, das opções de compilação e da configuração do ambiente de desenvolvimento.

Em muitos programas, é comum adotar a convenção de utilizar `<...>` para cabeçalhos da Biblioteca Padrão e `"..."` para cabeçalhos pertencentes ao próprio projeto. Essa convenção é útil e amplamente utilizada, mas não deve ser confundida com a definição das regras de localização de arquivos.

1.3.3 O Cabeçalho `<iostream>` e a Arquitetura de Streams

O cabeçalho `<iostream>` fornece as declarações necessárias para operações de entrada e saída baseadas em *streams* (fluxos). A Biblioteca Padrão do C++ utiliza a abstração de fluxo para representar operações de entrada e saída de dados de maneira independente do dispositivo específico envolvido [1, 2, 4].

Um *stream* pode ser entendido, em uma primeira aproximação, como uma abstração que representa uma sequência de dados que pode ser produzida por uma fonte ou consumida por um destino. Dessa forma, o mesmo modelo conceitual pode ser utilizado para trabalhar com diferentes dispositivos e fontes de dados, como o terminal, arquivos e outros componentes de entrada e saída.

O cabeçalho `<iostream>` disponibiliza, entre outros componentes, quatro objetos de fluxo associados aos fluxos padrão do programa: `std::cin`, `std::cout`, `std::cerr` e `std::clog` [1, 6].

A tabela 1.1 apresenta esses objetos e suas finalidades principais. Os fluxos `std::cout` e `std::cin` são, respectivamente, os principais objetos utilizados para saída e entrada de dados nos programas introdutórios deste material.

Tabela 1.1: Objetos de fluxo padrão disponibilizados por `<iostream>`.

Objeto	Fluxo	Finalidade
<code>std::cout</code>	Saída padrão	Utilizado para enviar dados para o fluxo de saída padrão, normalmente associado ao terminal ou console.
<code>std::cin</code>	Entrada padrão	Utilizado para receber dados provenientes do fluxo de entrada padrão, normalmente associado ao teclado ou a outra fonte configurada pelo ambiente de execução.
<code>std::cerr</code>	Erro padrão	Utilizado para enviar mensagens de erro e diagnóstico para o fluxo de erro padrão. Possui comportamento de bufferização distinto de <code>std::cout</code> e é destinado a mensagens que normalmente não fazem parte da saída regular do programa.
<code>std::clog</code>	Log padrão	Utilizado para mensagens de diagnóstico e registro. Assim como <code>std::cerr</code> , está associado ao fluxo de erro padrão, mas possui comportamento de bufferização apropriado para operações contínuas de registro.

Inclusão de Cabeçalhos Necessários

Cada arquivo-fonte deve incluir os cabeçalhos necessários para que as entidades utilizadas naquele arquivo sejam devidamente declaradas. Como boa prática, recomenda-se evitar inclusões que não sejam necessárias para a unidade de tradução atual, especialmente em projetos de maior porte.



Boa Prática de Programação 1.4

Inclua os cabeçalhos necessários para o código que está sendo compilado. Evitar inclusões desnecessárias reduz dependências entre arquivos e pode contribuir para diminuir os custos de pré-processamento e compilação em projetos maiores.

É importante, entretanto, não transformar essa recomendação em uma regra de que todo cabeçalho deve aparecer apenas uma única vez em todo o projeto. Cada arquivo-fonte ou cabeçalho deve declarar explicitamente as dependências necessárias ao seu próprio conteúdo, de acordo com a organização do projeto.

Cabeçalhos da Biblioteca C e seus Correspondentes em C++

A Biblioteca Padrão do C++ fornece cabeçalhos correspondentes a diversos componentes provenientes da biblioteca padrão da linguagem C. Em muitos casos, esses cabeçalhos possuem o prefixo `c`. Alguns exemplos são:

C++	Cabeçalho tradicional de C
<code><cmath></code>	<code><math.h></code>
<code><cstdio></code>	<code><stdio.h></code>
<code><cstdlib></code>	<code><stdlib.h></code>
<code><cstring></code>	<code><string.h></code>

Para programas escritos em C++, recomenda-se conhecer e utilizar os cabeçalhos especificados pela Biblioteca Padrão do C++, como `<cmath>`, `<cstdio>` e `<cstdlib>`, quando suas respectivas funcionalidades forem necessárias.



Boa Prática de Programação 1.5

Ao utilizar funcionalidades provenientes da biblioteca padrão da linguagem C em um programa C++, prefira, quando aplicável, os cabeçalhos correspondentes especificados pela Biblioteca Padrão do C++, como `<cmath>` e `<cstdio>`. A relação entre esses cabeçalhos e os correspondentes da linguagem C será aprofundada posteriormente, quando forem estudadas bibliotecas e *namespaces*.

Diretivas de Pré-processamento não utilizam ;

Diretivas de pré-processamento possuem uma sintaxe própria e não são instruções C++ convencionais. Portanto, a diretiva `#include` não deve ser seguida por um ponto e vírgula.

A forma correta é:

```
1 #include <iostream>
```

e não (forma incorreta):

```
1 #include <iostream>;
```

❗ Erro Comum de Programação 1.4: Ponto e vírgula após `#include`

Não coloque um ponto e vírgula ao final de uma diretiva `#include`. O caractere ; não faz parte da sintaxe da diretiva e poderá resultar em código inválido durante as etapas posteriores de tradução.

❗ Erro Comum de Programação 1.5: Interpretar `<...>` como sinônimo de Biblioteca Padrão

Não considere que a utilização de chaves angulares torna obrigatoriamente o arquivo um componente da Biblioteca Padrão. A escolha entre `<...>` e `"..."` está relacionada principalmente à estratégia de busca utilizada para localizar o arquivo. Diretórios adicionais podem ser configurados pelo sistema de compilação e pelo compilador.

1.4 Uso do Espaço de Nomes (*Namespace*) `std`

À medida que projetos de software crescem em tamanho e complexidade, torna-se cada vez mais importante organizar os nomes utilizados por diferentes partes do programa. Funções, classes, variáveis e outros identificadores podem possuir nomes iguais sem necessariamente representar a mesma entidade.

A linguagem C++ fornece o mecanismo de **espaços de nomes** (*namespaces*) para organizar identificadores e evitar conflitos entre nomes definidos em diferentes partes de

um programa [1, 4].

Um *namespace* introduz um contexto nomeado no qual entidades podem ser declaradas. Quando necessário, essas entidades podem ser acessadas por meio de seus nomes qualificados. Dessa forma, dois identificadores com o mesmo nome podem coexistir em espaços de nomes diferentes sem que isso represente, necessariamente, uma colisão entre eles.

1.4.1 O Espaço de Nomes da Biblioteca Padrão (std)

A Biblioteca Padrão do C++ disponibiliza seus componentes por meio do *namespace* `std`. Entre esses componentes encontram-se objetos de entrada e saída, como `std::cout` e `std::cin`, estruturas de dados, como `std::vector` e `std::string`, e algoritmos, como `std::sort` [1, 7].

Assim, quando escrevemos:

```
1 std::cout
```

estamos utilizando um **nome qualificado**. O identificador `cout` é especificado dentro do *namespace* `std` por meio do operador de resolução de escopo (`::`).

De maneira conceitual, podemos interpretar `std::cout` como uma especificação que indica que o identificador `cout` deve ser localizado no *namespace* `std`.

O mesmo mecanismo é utilizado para acessar outros componentes da Biblioteca Padrão:

```
1 std::cout
2 std::cin
3 std::string
4 std::vector
5 std::sort
```

A utilização explícita do nome qualificado torna a origem do identificador visível no próprio código e reduz a possibilidade de ambiguidades entre nomes provenientes de diferentes espaços de nomes.

1.4.2 Mecanismos de Qualificação e Uso da Diretiva `using`

O C++ oferece três abordagens distintas para interagir com elementos de um *namespace*. Cada estratégia apresenta desdobramentos diretos na segurança, clareza e manutenção do código:

1. **Qualificação Explícita (Recomendada):** utiliza-se o prefixo `std::` a cada invocação de um símbolo da biblioteca padrão.

```
1 std::cout << "Texto" << std::endl;
```

Vantagem: evita qualquer ambiguidades e deixa 100% explícito no código-fonte a origem de cada símbolo.

2. **Declaração de Uso Individual (*Using-Declaration*):** importa um símbolo específico do *namespace* para o escopo local ou global do arquivo.

```
1 using std::cout; // Torna 'cout' visível diretamente
2 using std::endl;
3
4 int main() {
5     cout << "Texto sem prefixo" << endl;
6     return 0;
7 }
```

Vantagem: reduz a verbosidade sem poluir o escopo com todos os centenas de símbolos contidos em `std`.

3. **Diretiva de Uso Global (*Using-Directive*):** disponibiliza **todos** os símbolos contidos no *namespace* `std` para o escopo atual.

```
1 using namespace std; // Importa TODO o namespace std
2
3 int main() {
4     cout << "Uso direto de cout" << endl;
5     return 0;
6 }
```

Risco: elimina a proteção contra colisão de nomes, tornando o código vulnerável a ambiguidades conforme bibliotecas adicionais são incluídas.

1.4.3 Perigo do `using namespace std;` em Arquivos de Cabeçalho (.h / .hpp)

A diretiva `using namespace` permite que os nomes pertencentes a um *namespace* sejam considerados durante a busca por nomes em determinados contextos, sem que seja necessário utilizar sua qualificação completa. Embora esse recurso possa ser conveniente em programas pequenos, seu uso em escopo global de arquivos de cabeçalho deve ser evitado [1, 7, 4].

Considere, por exemplo, um arquivo de cabeçalho que contenha:

```
1 #ifndef MINHA_BIBLIOTECA_HPP
2 #define MINHA_BIBLIOTECA_HPP
3
4 #include <string>
5
6 using namespace std;
7
8 class Pessoa
9 {
10     string nome;
11 };
12
13 #endif
```

Quando esse cabeçalho é incluído por outro arquivo por meio de `#include`, a diretiva `using namespace std;` passa a fazer parte do contexto de compilação daquele arquivo de inclusão. Como os cabeçalhos são processados durante o pré-processamento, essa decisão pode afetar outros componentes que, direta ou indiretamente, incluam o cabeçalho.

Esse efeito é particularmente problemático em projetos maiores, nos quais diferentes bibliotecas e componentes podem utilizar identificadores iguais. A introdução de uma *using-directive* em um cabeçalho reduz a localidade da decisão sobre quais nomes estarão disponíveis sem qualificação e pode aumentar a possibilidade de ambiguidades e conflitos de nomes [7, 4].

👍 Boa Prática de Programação 1.6

Evite utilizar `using namespace std;` em escopo global de arquivos de cabeçalho. Prefira a qualificação explícita, como `std::cout`, `std::string` e `std::vector`. Dessa forma, o código deixa explícita a origem dos identificadores utilizados e reduz a possibilidade de interferência entre diferentes componentes do projeto.

Uma alternativa mais localizada consiste em utilizar uma *using-declaration* para introduzir somente o nome necessário:

```
1 using std::string;
```

Mesmo nesse caso, a localização da declaração deve ser considerada cuidadosamente. Em particular, uma declaração colocada em escopo global de um cabeçalho também pode afetar o código que inclui esse cabeçalho. Por essa razão, em interfaces de cabeçalho, a qualificação explícita `std::string` é geralmente a alternativa mais clara.

🚫 Erro Comum de Programação 1.6

Utilizar `using namespace std;` em escopo global dentro de um arquivo de cabeçalho e presumir que seus efeitos ficarão restritos ao próprio arquivo. Como o cabeçalho pode ser incluído por diversos arquivos-fonte, uma *using-directive* nesse contexto pode influenciar a resolução de nomes em código que não contém explicitamente a diretiva.

Ambiguidades provocadas pela busca de nomes

Um dos problemas associados ao uso indiscriminado de `using namespace` é a possibilidade de introduzir nomes adicionais no contexto em que ocorre a busca por identificadores.

Considere um exemplo simplificado:

```
1 namespace BibliotecaA
2 {
3     void imprimir();
4 }
```

```
5
6 namespace BibliotecaB
7 {
8     void imprimir();
9 }
10
11 using namespace BibliotecaA;
12 using namespace BibliotecaB;
13
14 int main()
15 {
16     imprimir(); // Ambiguidade
17     return 0;
18 }
```

Nesse caso, a chamada `imprimir()` não fornece informação suficiente para determinar qual das duas funções deve ser utilizada. O compilador pode, portanto, diagnosticar uma chamada ambígua.

A solução consiste em qualificar explicitamente o nome:

```
1 BibliotecaA::imprimir();
2 BibliotecaB::imprimir();
```

O mesmo princípio se aplica à Biblioteca Padrão. Ao utilizar `std::string`, por exemplo, o programador deixa explícito que deseja o tipo `string` pertencente ao *namespace* `std`.

❗ Erro Comum de Programação 1.7

Não presuma que a simples existência de dois identificadores com o mesmo nome produzirá necessariamente um erro de ambiguidade. O resultado depende do contexto, das regras de escopo e da busca de nomes. O problema central do uso indiscriminado de `using namespace` é ampliar implicitamente o conjunto de nomes considerados durante essa busca.

1.4.4 Identificadores da Biblioteca Padrão e Escopo

Um ponto importante para compreender o uso do *namespace* `std` é distinguir entre **palavras-chave da linguagem** e **identificadores disponibilizados pela Biblioteca Padrão**. Embora nomes como `cin`, `cout`, `vector` e `string` sejam amplamente utilizados em programas C++, eles não são palavras reservadas da linguagem.

Por exemplo, considere o seguinte programa:

Código 1.2: `code_02.cpp`

```

1 #include <iostream>
2 int main()
3 {
4     std::cout << "Antes da declaracao de cin.\n";
5     int cin = 10;
6     std::cout << cin << '\n';
7
8     return 0;
9 }
```

Esse programa é válido e a declaração:

```

1 int cin = 10;
```

não entra em conflito direto com `std::cin`. Isso ocorre porque os dois nomes designam entidades diferentes e pertencem a contextos diferentes.

O objeto de entrada padrão é identificado pelo nome qualificado `std::cin`, enquanto a variável declarada pelo programador é identificada simplesmente como `cin`.

Assim, podemos representar conceitualmente a situação da seguinte maneira:

Nome	Entidade	Contexto
<code>std::cin</code>	Objeto de entrada padrão	namespace <code>std</code>
<code>cin</code>	Variável do tipo <code>int</code>	Bloco de <code>main</code>

A diferença torna-se ainda mais evidente quando os dois nomes são utilizados explicitamente no mesmo programa:

Código 1.3: code_03.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     int cin = 10;
6     std::cout << "Valor da variavel cin: ";
7     std::cout << cin << '\n';
8
9     std::cout << "Entrada padrao: ";
10    std::string entrada;
11    std::cin >> entrada;
12    std::cout << "Voce digitou: " << entrada << '\n';
13
14    return 0;
15 }
```

Nesse exemplo, `cin` e `std::cin` continuam representando entidades distintas. A qualificação `std::` deixa explícito que o segundo identificador deve ser procurado no *namespace* da Biblioteca Padrão.

Esse comportamento demonstra por que é importante não utilizar a expressão “nome reservado” de maneira genérica. Em C++, uma palavra-chave da linguagem e um identificador pertencente à Biblioteca Padrão são conceitos distintos.

Portanto, as seguintes categorias apresentadas na Tabela (1.2) devem ser diferenciadas:

Tabela 1.2: Distinções entre palavras-chave, identificadores e nomes qualificados em C++.

Conceito	Exemplo	Significado
Palavra-chave	<code>int</code> , <code>return</code> , <code>class</code>	Palavra reservada pela linguagem C++ para uma finalidade sintática ou semântica específica. Não pode ser utilizada livremente como identificador.
Continua na próxima página		

Tabela 1.2 – Continuação da página anterior

Conceito	Exemplo	Significado
Identificador da Biblioteca Padrão	<code>std::cin,</code> <code>std::cout,</code> <code>std::vector</code>	Nome de uma entidade disponibilizada pela Biblioteca Padrão, normalmente pertencente ao <i>namespace</i> <code>std</code> .
Identificador local	<code>int cin = 10;</code>	Nome definido pelo programador em determinado contexto ou escopo, sujeito às regras de declaração e busca de nomes.
Nome qualificado	<code>std::cin</code>	Nome acompanhado da especificação do <i>namespace</i> ou escopo ao qual pertence.
Nome não qualificado	<code>cin</code>	Nome utilizado sem especificação explícita de seu <i>namespace</i> ou escopo. Sua interpretação depende das regras de busca de nomes.
Ocultação de nome	<code>int cout = 10;</code>	Situação em que um nome declarado em um escopo mais interno pode ocultar outro nome visível proveniente de um escopo externo.

Exemplo de Ocultação de Nome

Embora a declaração de uma variável chamada `cin` seja válida, a situação pode tornar-se problemática quando nomes do *namespace* `std` são introduzidos no contexto sem sua qualificação explícita. Considere o próximo código.

Código 1.4: code_04.cpp

```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int cout = 10;
7      cout << "Ola\n";
8
9      return 0;
10 }
```


⚠ Erro Comum de Programação 1.8

Não considere nomes como `cin`, `cout`, `string` ou `vector` como palavras-chave da linguagem C++. Esses nomes são identificadores de entidades da Biblioteca Padrão. O fato de normalmente serem utilizados com o prefixo `std::` não os transforma em palavras reservadas.

1.5 Entrada e Saída de Dados com `std::cout` e `std::cin`

A interação com o usuário em aplicações de console C++ pode ser realizada por meio da biblioteca de entrada e saída (*Input/Output Library*), baseada no conceito de **fluxos** (*streams*). Um fluxo fornece uma abstração para operações de entrada e saída, permitindo que o programa transfira dados entre suas estruturas internas e uma fonte ou destino associado ao fluxo [4, 6, 8].

Entre os objetos de fluxo disponibilizados pela Biblioteca Padrão encontram-se `std::cout`, utilizado para a saída padrão, e `std::cin`, utilizado para a entrada padrão. Em condições usuais de execução de um programa de console, a saída padrão está associada ao terminal e a entrada padrão está associada à entrada fornecida pelo usuário. Entretanto, essa associação não deve ser confundida com uma propriedade intrínseca dos objetos: os fluxos são abstrações que podem ser redirecionadas pelo ambiente de execução [9, 10].

Nesta seção, serão apresentados os mecanismos fundamentais de entrada e saída utilizados em programas C++. Serão estudados os objetos `std::cout` e `std::cin`, os operadores de inserção e extração, o encadeamento de operações, os caracteres de nova linha, o manipulador `std::endl` e a leitura de linhas completas com `std::getline`.

1.5.1 Saída de Dados com `std::cout`

O objeto `std::cout` é um objeto global da Biblioteca Padrão do C++ associado ao fluxo de saída padrão. Seu tipo é `std::ostream`, especialização de `std::basic_ostream<char>`. A saída formatada é realizada, em geral, por meio do operador de inserção ‘`<<`’ [9, 11].

É importante observar que `std::cout` representa o fluxo de saída padrão do programa. Em uma execução convencional, esse fluxo pode estar conectado ao terminal, mas também pode ser redirecionado para outro destino pelo sistema operacional ou pelo ambiente de execução.

Conforme vimos no Código (1.1), a expressão

```
1 std::cout << "Bem-vindo ao C++!\n";
```

realiza uma operação de inserção no fluxo de saída. A cadeia de caracteres é inserida no fluxo controlado por `std::cout`.

O Operador de Inserção «

O operador ‘«’, quando utilizado com um fluxo de saída, é denominado **operador de inserção**. A Biblioteca Padrão fornece sobrecargas desse operador para diversos tipos fundamentais e para várias classes da própria biblioteca. Além disso, tipos definidos pelo programador podem fornecer suas próprias sobrecargas de `operator «` [12].

Consequentemente, o mecanismo não deve ser entendido como uma simples operação textual. O valor fornecido ao operador é interpretado de acordo com seu tipo e com a sobrecarga correspondente. Por exemplo, no código a seguir, os valores de tipos diferentes são inseridos no mesmo fluxo.

Código 1.5: code_05.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     int idade = 25;
6     double altura = 1.75;
7     char inicial = 'W';
8     std::cout << "Idade: " << idade << '\n';
9     std::cout << "Altura: " << altura << '\n';
10    std::cout << "Inicial: " << inicial << '\n';
11    return 0;
12 }
```

A Biblioteca Padrão possui as operações necessárias para representar esses valores na saída formatada.

Encadeamento do Operador de Inserção

Uma característica importante do operador de inserção é que ele retorna uma referência ao próprio fluxo. Essa propriedade permite encadear várias operações de inserção em uma única expressão [11].

Considere:

Código 1.6: code_06.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     int idade = 25;
6     double altura = 1.75;
7     std::cout << "O usuario tem " << idade
8         << " anos e mede " << altura << " m de altura.\n";
9
10    return 0;
11 }
```

A expressão pode ser entendida conceitualmente como uma sequência de operações sobre o mesmo fluxo:

```
1 std::cout << "O usuario tem ";
2 std::cout << idade;
3 std::cout << " anos e mede ";
4 std::cout << altura;
5 std::cout << " m de altura.\n";
```

A forma encadeada é, portanto, uma maneira compacta de realizar várias operações sucessivas sobre o fluxo.

Nova Linha: `\n` e `std::endl`

Existem diferentes maneiras de produzir uma nova linha na saída. Duas formas comuns nos programas introdutórios são o caractere de nova linha `\n` e o manipulador `std::endl`.

O caractere de nova linha pode ser inserido diretamente no fluxo:

```
1 // Uso do caractere de nova linha
2 std::cout << "Primeira linha\n";
3 std::cout << "Segunda linha\n";
```

Já o manipulador `std::endl` realiza duas operações: insere um caractere de nova linha no fluxo e solicita a descarga (*flush*) do fluxo [13].

```
1 std::cout << "Primeira linha" << std::endl;
2 std::cout << "Segunda linha" << std::endl;
```

Portanto, embora ambos produzam uma nova linha, não são semanticamente idênticos:

Construção	Comportamento
<code>\n</code>	Insere um caractere de nova linha no fluxo. Não solicita, por si só, uma descarga explícita do fluxo.
<code>std::endl</code>	Insere um caractere de nova linha e realiza uma operação de <i>flush</i> no fluxo.

O *flush* é uma operação que solicita que os dados pendentes no fluxo sejam transferidos para a sequência de saída subjacente. O custo dessa operação depende do ambiente e do destino do fluxo [13, 14].

Em programas que realizam grande quantidade de operações de saída, utilizar `std::endl` indiscriminadamente pode introduzir operações de *flush* desnecessárias. Por essa razão, quando a única intenção é produzir uma nova linha, `\n` geralmente é a alternativa preferível [13].

1.5.2 Entrada de Dados com `std::cin`

O `std::cin` é um objeto global da Biblioteca Padrão associado ao fluxo de entrada padrão. Seu tipo é `std::istream`, especialização de `std::basic_istream<char>`. A entrada formatada pode ser realizada por meio do operador de extração `operator>>` [10, 15].

Um exemplo elementar, demonstrando a forma de entrada de um valor inteiro, é apresentado no código a seguir.

Código 1.7: `code_07.cpp`

```
1 #include <iostream>
2
3 int main()
4 {
5     int idade;
6     std::cout << "Digite sua idade: ";
7     std::cin >> idade;
8     std::cout << "Idade informada: " << idade << '\n';
9
10    return 0;
11 }
```

Nesse caso, o operador `std::cin >> idade;` realiza uma operação de extração do fluxo de entrada e tenta interpretar os dados de acordo com o tipo da variável `idade`.

Entrada Formatada e Espaços em Branco

O operador `>>`, quando utilizado para entrada formatada, segue regras específicas para cada tipo de destino. Para diversos tipos de entrada formatada, os espaços em branco iniciais são ignorados antes da extração do valor. Entre os caracteres considerados espaços em branco estão espaços, tabulações e quebras de linha [15].

O código a seguir apresenta um exemplo de extração de forma sequencial, através do uso do `std::cin`. É esperado um espaço em branco entre os valores, para que sejam instanciados nas variáveis encadeadas declaradas.

Código 1.8: code_08.cpp

```
1 // Leitura de dois valores
2 #include <iostream>
3
4 int main()
5 {
6     int x;
7     double y;
8
9     std::cout << "Digite um numero inteiro e um numero decimal: ";
10    std::cin >> x >> y;
11
12    std::cout << "x = " << x << '\n';
13    std::cout << "y = " << y << '\n';
14    return 0;
15 }
```

Se o usuário fornecer:

```
10 3.14
```

as duas operações de extração serão realizadas sequencialmente. O primeiro valor será extraído para x e o segundo para y. O encadeamento

```
1 std::cin >> x >> y;
```

possui comportamento análogo ao encadeamento observado anteriormente com `std::cout`: cada operação retorna uma referência ao fluxo, permitindo que outra operação seja realizada sobre o mesmo objeto.

Um padrão fundamental em programas introdutórios consiste na separação conceitual entre três etapas:

1. **Entrada:** obtenção dos dados necessários para o processamento;
2. **Processamento:** realização dos cálculos ou transformações;
3. **Saída:** apresentação dos resultados.

Esse modelo é frequentemente denominado ciclo **entrada-processamento-saída** e constitui uma abstração útil para compreender programas simples. O próximo tópico exemplifica essas etapas.

1.5.3 Exemplo Prático: Cálculo de Média

O programa a seguir reúne os conceitos apresentados até aqui. O usuário fornece duas notas, o programa calcula a média aritmética e, posteriormente, apresenta o resultado.

Código 1.9: code_09.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     // Entrada
6     double nota1 = 0.0;
7     double nota2 = 0.0;
8
9     std::cout << "=== SISTEMA DE CALCULO DE MEDIA ===\n";
10    std::cout << "Digite a primeira nota: ";
11    std::cin >> nota1;
12
13    std::cout << "Digite a segunda nota: ";
14    std::cin >> nota2;
15
16    // Processamento
17    double media = (nota1 + nota2) / 2.0;
18
19    // Saida
20    std::cout << "\n--- Resultado Final ---\n";
21    std::cout << "Nota 1: " << nota1 << '\n';
22    std::cout << "Nota 2: " << nota2 << '\n';
23    std::cout << "Media apurada: " << media << '\n';
24
25    return 0;
26 }
```

Esse exemplo também evidencia uma característica importante do operador `»`: a interpretação dos dados depende do tipo da variável de destino. Nesse caso, as variáveis `nota1`, `nota2` e `media` são do tipo `double`, permitindo a representação de valores reais de acordo com as regras desse tipo.

É importante destacar, entretanto, que a utilização de `std::cin` `»` não garante que o usuário tenha fornecido uma entrada válida. Se o texto fornecido não puder ser interpretado de acordo com o tipo esperado, o fluxo poderá entrar em estado de falha. O tratamento sistemático desses estados será estudado posteriormente.

1.5.4 Leitura de Textos com Espaços: `std::getline`

O operador de extração formatada `»` e a função `std::getline` possuem objetivos diferentes.

Quando utilizado para a leitura de tipos como inteiros ou números de ponto flutuante, o operador `»` procura interpretar uma sequência de entrada de acordo com o tipo solicitado. Já `std::getline` realiza uma leitura orientada a linhas: extrai caracteres do fluxo até encontrar o delimitador especificado. Na forma mais comum, o delimitador padrão é o caractere de nova linha `'\n'` [16].

Para utilizar `std::string` e `std::getline`, deve-se incluir o cabeçalho correspondente:

Código 1.10: `code_10.cpp`

```
1 // Leitura de uma linha completa
2 #include <iostream>
3 #include <string>
4
5 int main()
6 {
7     std::string nome_completo;
8     std::cout << "Digite seu nome completo: ";
9     std::getline(std::cin, nome_completo);
10    std::cout << "Bem-vindo, " << nome_completo << "!\n";
11    return 0;
12 }
```

Se o usuário digitar:

```
Maria da Silva
```

a variável `nome_completo` receberá a linha contendo os espaços, enquanto uma operação convencional de extração para uma cadeia de caracteres delimitada por espaço teria comportamento diferente.

Uma Situação Importante: `std::cin` » seguido de `std::getline`

Um dos problemas mais frequentes nos primeiros programas de entrada de dados ocorre quando uma operação de extração formatada é imediatamente seguida por uma chamada a `std::getline`.

Considere:

Código 1.11: `code_11.cpp`

```
1 // Problema ao combinar cin e getline
2 #include <iostream>
3 #include <string>
4
5 int main()
6 {
7     int idade;
8     std::string nome;
9
10    std::cout << "Digite sua idade: ";
11    std::cin >> idade;
12
13    std::cout << "Digite seu nome: ";
14    std::getline(std::cin, nome);
15
16    std::cout << "Nome: " << nome << '\n';
17
18    return 0;
19 }
```

Suponha que o usuário digite:

25

e pressione Enter. A extração de idade obtém o valor 25, mas o delimitador que encerra a entrada permanece no fluxo. Quando `std::getline` é chamada imediatamente em seguida, ela encontra esse delimitador e pode produzir uma linha vazia [16].

Uma solução comum consiste em descartar o restante da linha antes de chamar `std::getline`. Para isso, utilizaremos também o cabeçalho `<limits>`, que fornece a classe de utilidade `std::numeric_limits`. Neste exemplo, ela será utilizada para obter o maior valor representável pelo tipo `std::streamsize`, permitindo solicitar ao fluxo que descarte uma quantidade suficientemente grande de caracteres até encontrar o delimitador de fim de linha.

Código 1.12: code_12.cpp

```
1 // Tratamento do restante da linha
2 #include <iostream>
3 #include <string>
4 #include <limits>
5
6 int main()
7 {
8     int idade;
9     std::string nome;
10
11     std::cout << "Digite sua idade: ";
12     std::cin >> idade;
13     std::cin.ignore(
14         std::numeric_limits<std::streamsize>::max(), '\n'
15     );
16     std::cout << "Digite seu nome: ";
17     std::getline(std::cin, nome);
18     std::cout << "Nome: " << nome << '\n';
19     return 0;
20 }
```

Outra possibilidade, dependendo da situação, é utilizar o manipulador `std::ws` para consumir espaços em branco antes da leitura da linha. A escolha da estratégia depende do comportamento de entrada desejado [16].

1.5.5 Relação entre `std::cin`, `std::cout` e o Fluxo de Execução

Em aplicações interativas, é comum apresentar uma mensagem ao usuário e, em seguida, solicitar uma entrada:

```
1 std::cout << "Digite um valor: ";  
2  
3 int valor;  
4 std::cin >> valor;
```

Existe ainda uma relação importante entre os dois objetos. Por especificação, `std::cin` está associado (*tied*) a `std::cout` por meio do mecanismo de *tie*. Assim, uma operação de entrada em `std::cin` pode provocar a descarga de `std::cout` antes da leitura, garantindo que uma mensagem de solicitação pendente seja apresentada antes que o programa aguarde a entrada do usuário [10].

Isso significa que, em uma situação típica como

```
1 std::cout << "Digite sua idade: ";  
2 std::cin >> idade;
```

não é necessário utilizar obrigatoriamente `std::endl` apenas para garantir que o *prompt* seja exibido antes da leitura. O vínculo entre os fluxos fornece esse comportamento em condições normais.

1.5.6 Formatação de Texto com `std::format` em C++20

A partir do padrão C++20, a Biblioteca Padrão passou a disponibilizar uma infraestrutura própria para formatação de texto por meio do cabeçalho `<format>`. Entre os recursos disponibilizados está a função `std::format`, que recebe uma cadeia de formato e um conjunto de argumentos e retorna uma nova `std::string` contendo o texto formatado

[17].

Diferentemente de uma operação direta de saída com `std::cout`, `std::format` não escreve diretamente no terminal. Sua finalidade é **construir uma cadeia de caracteres formatada**. Essa cadeia pode, posteriormente, ser armazenada em uma variável, manipulada pelo programa ou enviada para um fluxo de saída, como `std::cout`.

O exemplo a seguir demonstra esse mecanismo:

Código 1.13: code_13.cpp

```
1 #include <iostream>
2 #include <format>
3 #include <string>
4
5 int main()
6 {
7     int id = 42;
8     double pi = 3.14159;
9
10    std::string msg = std::format("ID: {}, PI: {:.2f}", id, pi);
11    std::cout << msg << '\n';
12    return 0;
13 }
```

Nesse exemplo, a expressão

```
1 std::format("ID: {}, PI: {:.2f}", id, pi)
```

produz uma cadeia de caracteres a partir dos valores armazenados em `id` e `pi`. Os pares de chaves representam campos de substituição. O primeiro campo, `{}`, é substituído pelo valor de `id`. O segundo campo, `{:.2f}`, especifica que o valor de `pi` deve ser apresentado com duas casas decimais.

Assim, considerando os valores utilizados no exemplo, o conteúdo produzido por `std::format` será equivalente a:

```
ID: 42, PI: 3.14
```

A cadeia resultante é armazenada na variável `msg`, cujo tipo é `std::string`. Somente posteriormente essa cadeia é enviada para a saída padrão por meio de `std::cout`.

Essa separação entre **formatação** e **saída** é conceitualmente importante. A função `std::format` produz um objeto textual; ela não possui, por si só, a responsabilidade de exibir esse texto no terminal [17].

Requisito do padrão C++20

A função `std::format` pertence à Biblioteca Padrão a partir do C++20. Portanto, um programa que utilize esse recurso deve ser compilado com um compilador e uma biblioteca padrão que ofereçam suporte correspondente ao C++20 [17].

Ao utilizar o GCC, por exemplo, o padrão da linguagem pode ser explicitamente selecionado por meio da opção `-std=c++20`:

```
g++ -std=c++20 code_13.cpp -o code_13
```

A execução do programa pode então ser realizada por meio de:

```
./code_13
```

No Windows, utilizando um ambiente em que o executável seja produzido com a extensão `.exe`, a sequência correspondente pode ser:

```
g++ -std=c++20 code_13.cpp -o code_13.exe
```

cujo comando de execução será:

```
code_13.exe
```

Configuração do padrão C++ no Visual Studio Code

Editores e ambientes de desenvolvimento precisam conhecer o padrão da linguagem utilizado pelo projeto para fornecer recursos como análise sintática, realce de código, diagnóstico e IntelliSense.

No Visual Studio Code, quando utilizada a extensão oficial de C/C++, é possível configurar o padrão da linguagem por meio das configurações da própria extensão. Uma forma simples de realizar essa configuração consiste em utilizar o arquivo `settings.json`.

Para abrir esse arquivo, pressione `Ctrl + Shift + P` e execute o comando *Preferences: Open User Settings (JSON)*. Em uma configuração específica do projeto, também é possível utilizar as configurações do *workspace*.

No arquivo `settings.json`, adicione a seguinte propriedade:

```
"C_Cpp.default.cppStandard": "c++20"
```

A propriedade `C_Cpp.default.cppStandard` define o padrão C++ utilizado pela extensão C/C++ como configuração padrão do IntelliSense. Assim, recursos como análise sintática, diagnóstico e sugestões de código passam a considerar as regras correspondentes ao padrão C++20.

Também é possível utilizar outros padrões suportados pela extensão, como `c++17` e `c++23`, desde que o compilador utilizado pelo projeto ofereça suporte ao padrão correspondente.

Boa Prática de Programação 1.8

Configure o padrão da linguagem de acordo com o padrão efetivamente utilizado pelo projeto. Se os exemplos deste material forem compilados com C++20, mantenha a configuração do IntelliSense em `c++20`.

Assim, existem duas configurações conceitualmente distintas:

- **Configuração do ambiente de desenvolvimento:** informa ao editor e ao IntelliSense quais regras da linguagem devem ser consideradas;

- **Configuração do compilador:** determina efetivamente quais regras e recursos do padrão serão utilizados durante a compilação.

👍 Boa Prática de Programação 1.9

Quando a intenção for simplesmente produzir uma nova linha, prefira `\n` a `std::endl`. Reserve `std::endl` para situações nas quais a descarga explícita do fluxo seja realmente necessária. O uso indiscriminado de `std::endl` pode introduzir operações de *flush* desnecessárias e prejudicar o desempenho em aplicações com grande volume de saída [13].

👍 Boa Prática de Programação 1.10

Mantenha separados, sempre que possível, os conceitos de entrada, processamento e saída. Essa organização torna o fluxo lógico do programa mais fácil de compreender, testar e posteriormente modificar.

👍 Boa Prática de Programação 1.11

Utilize `std::getline` quando a unidade lógica de entrada for uma linha completa de texto. Utilize `std::cin >>` quando a intenção for realizar entrada formatada de valores ou tokens.

⊗ Erro Comum de Programação 1.9

Inverter os operadores de fluxo. Para saída, utilize o operador de inserção `<<`; para entrada, utilize o operador de extração `>>`.

```
1 // Direcao dos operadores de fluxo
2 std::cout << valor; // Saída
3 std::cin >> valor; // Entrada
```

A interpretação conceitual é que, na primeira expressão, o valor é inserido no fluxo de saída, enquanto, na segunda, um valor é extraído do fluxo de entrada para a variável.

❗ Erro Comum de Programação 1.10

Utilizar `std::cin` » imediatamente antes de `std::getline` sem considerar os caracteres que permanecem no fluxo. A operação de extração formatada pode deixar o delimitador que encerrou a entrada, fazendo com que `std::getline` o encontre imediatamente e produza uma linha vazia [16].

❗ Erro Comum de Programação 1.11

Assumir que uma operação como

```
1 std::cin >> idade;
```

garante que `idade` recebeu uma entrada válida. Operações de entrada podem falhar quando os dados fornecidos não são compatíveis com o tipo esperado. O tratamento dos estados `goodbit`, `failbit`, `eofbit` e `badbit` será estudado posteriormente.

❗ Erro Comum de Programação 1.12: Confundir a configuração do VS Code com a configuração do compilador

Configurar `"cppStandard": "c++20"` no Visual Studio Code não garante, por si só, que o compilador utilizará o padrão C++20. Essa propriedade orienta a extensão C/C++ e o mecanismo de IntelliSense na interpretação do código.

Quando a compilação é realizada diretamente pelo GCC, por exemplo, o padrão deve ser especificado no comando de compilação por meio de `-std=c++20`. Em projetos maiores, essa configuração normalmente é gerenciada pelo sistema de construção utilizado pelo projeto.

👍 Boa Prática de Programação 1.12

Ao utilizar recursos introduzidos por uma nova versão do padrão C++, indique explicitamente o padrão exigido pelo programa no sistema de compilação. Isso torna a dependência do código em relação ao padrão da linguagem explícita e reduz problemas de portabilidade e configuração do ambiente.

1.6 Declaração, Inicialização e Atribuição de Variáveis

Em C++, os dados manipulados por um programa são associados a entidades definidas segundo o sistema de tipos da linguagem. Entre essas entidades estão as **variáveis**, que permitem ao programa armazenar e manipular valores durante sua execução.

Uma variável possui um **nome**, um **tipo** e, conforme o contexto, uma determinada duração de armazenamento. O tipo associado à variável determina quais valores podem ser representados e quais operações podem ser realizadas sobre o objeto por ela designado.

Para compreender corretamente o uso de variáveis em C++, é importante distinguir três conceitos que frequentemente são confundidos por iniciantes: **declaração**, **inicialização** e **atribuição**.

A declaração introduz uma entidade em determinado contexto e informa ao compilador as propriedades necessárias para que ela possa ser utilizada. A inicialização estabelece o valor inicial de um objeto no momento em que ele é criado. Já a atribuição modifica o valor de um objeto que já existe.

Essa distinção é fundamental porque inicialização e atribuição são operações diferentes em C++, embora possam produzir resultados visualmente semelhantes em exemplos simples.

1.6.1 Declaração de Variáveis e Tipagem

Uma declaração de variável especifica, entre outras informações, seu tipo e seu identificador. Uma forma elementar de declaração é:

```
1 int quantidade;
```

Nesse exemplo, `int` especifica o tipo da variável e `quantidade` é o identificador utilizado para referenciá-la.

É importante observar que o termo **declaração** possui um significado mais amplo na linguagem C++. Uma declaração pode introduzir diferentes tipos de entidades, como

variáveis, funções, classes e outros elementos da linguagem. Nesta seção, restringiremos inicialmente a discussão às declarações de variáveis.

O tipo da variável estabelece as regras que determinam os valores que podem ser armazenados no objeto e as operações que podem ser realizadas sobre ele. Por exemplo, observe o bloco a seguir:

```
1 // Variaveis de tipos diferentes
2 int idade;
3 double altura;
4 char inicial;
5 bool ativo;
```

Veja que as quatro variáveis possuem tipos diferentes e, conseqüentemente, possuem diferentes conjuntos de valores e operações que poderão ser associadas.

Aviso

O tamanho ocupado por um objeto não deve ser apresentado simplesmente como uma quantidade fixa de bytes determinada pelo nome do tipo. A representação e o tamanho dos tipos são definidos pelas regras da linguagem e pela implementação utilizada. Quando necessário, o tamanho de um objeto pode ser consultado por meio do operador `sizeof`.

1.6.2 Variáveis Locais sem Inicialização

Uma variável local automática que seja definida sem uma inicialização explícita não recebe automaticamente um valor útil definido pelo programador.

No caso do bloco a seguir:

```
1 int quantidade;
```

a variável `quantidade` foi definida, mas nenhum valor inicial foi especificado pelo programa.

Para variáveis locais de tipos fundamentais, como `int`, `double` e `char`, não se deve presumir que a variável contenha zero ou qualquer outro valor previsível. O conteúdo inicial não deve ser tratado como um valor válido produzido pelo programa.

É comum utilizar a expressão informal *lixo de memória* para descrever essa situação. Entretanto, a terminologia tecnicamente mais adequada é **valor indeterminado**.

❗ Erro Comum de Programação 1.13

Não leia uma variável local antes que ela tenha sido inicializada com um valor válido. O fato de uma determinada execução aparentemente produzir um número “plausível” não significa que esse resultado seja garantido pela linguagem.

Por exemplo, o código abaixo não deve ser utilizado para tentar descobrir qual valor estava anteriormente armazenado naquela região de memória:

```
1 // Leitura indevida de variavel nao inicializada
2 int quantidade;
3
4 std::cout << quantidade << '\n';
```

O programador não deve interpretar o resultado dessa operação como um valor confiável. A abordagem correta é fornecer explicitamente um valor inicial quando a variável precisar ser utilizada.

1.6.3 Inicialização de Variáveis

A **inicialização** é a operação pela qual um objeto recebe seu valor inicial no momento de sua criação. Em C++, existem diferentes formas sintáticas de inicialização. As principais formas utilizadas em programas são a inicialização por cópia, a inicialização direta e a inicialização por lista.

```
1 int a = 10;           // Inicializacao por copia
2 int b(20);           // Inicializacao direta
3 int c{30};           // Inicializacao por lista
```

Embora essas formas possam produzir resultados equivalentes em exemplos simples, elas não são semanticamente idênticas em todos os contextos.

Para código moderno em C++, a inicialização por lista, utilizando chaves, é particularmente importante porque fornece regras mais restritivas de conversão de tipos.

```
1 int quantidade{10};
2 double media{8.5};
3 char inicial{'W'};
```

A inicialização por lista também pode impedir determinadas conversões implícitas que poderiam resultar em perda de informação:

```
1 // Conversao potencialmente perigosa
2 double valor{3.14};
3
4 // O codigo abaixo nao e aceito:
5 // int numero{3.14};
```

Esse comportamento torna a inicialização por lista uma alternativa interessante para código moderno, especialmente quando se deseja que determinadas conversões sejam identificadas pelo compilador.

1.6.4 Inicialização versus Atribuição

Uma das distinções mais importantes desta seção é a diferença entre **inicialização** e **atribuição**.

Considere:

```
1 int idade = 25;
```

Nesse caso, a variável é criada e recebe seu valor inicial de 25. Agora considere:

```
1 int idade = 25;
2 idade = 30;
```

A primeira operação é uma inicialização. A segunda é uma atribuição: o objeto `idade`, que já existia, recebe um novo valor.

Podemos representar a diferença conceitualmente da seguinte maneira:

- **Inicialização:** criação do objeto acompanhada do estabelecimento de seu valor inicial;

- **Atribuição:** alteração do valor de um objeto que já existe.

Essa distinção torna-se ainda mais importante quando estudarmos tipos de classes, construtores, referências e outros mecanismos da linguagem.

Boa Prática de Programação 1.13

Inicialize as variáveis no momento de sua definição sempre que houver um valor inicial apropriado. Além de tornar a intenção do código explícita, essa prática reduz a possibilidade de utilização acidental de objetos sem um valor válido.

1.6.5 Dedução Automática de Tipos com `auto` (C++11)

A partir do padrão C++11, a palavra-chave `auto` permite que o compilador **deduza o tipo de uma variável a partir de seu inicializador**. Dessa forma, o programador não precisa escrever explicitamente o tipo quando ele puder ser determinado pelas regras de dedução da linguagem.

Por exemplo:

```
1 auto limite = 100;  
2 auto taxa = 0.085;  
3 auto mensagem = "Texto";
```

Nesse exemplo, o compilador deduz, respectivamente, os tipos `int`, `double` e `const char*` para as três variáveis.

É importante compreender que o uso de `auto` **não elimina o sistema de tipos de C++**. A variável continua possuindo um tipo determinado em tempo de compilação. O que muda é apenas a forma como esse tipo é especificado no código-fonte.

Assim, as duas declarações seguintes produzem variáveis do mesmo tipo:

```
1 int quantidade = 100;  
2 auto quantidade = 100;
```

Na primeira forma, o programador especifica explicitamente o tipo `int`. Na segunda, o compilador deduz esse tipo a partir do inicializador `100`.

NOTA

O `auto` não representa uma tipagem dinâmica. Uma vez deduzido o tipo da variável durante a compilação, esse tipo não muda durante a execução do programa.

Por exemplo:

```
1 auto valor = 10;
2 valor = 20;           // Correto: 20 também é um int
3 // valor = 3.14;      // A variável continua sendo do tipo int
```

Nesse caso, `valor` é deduzida como sendo do tipo `int`. A atribuição posterior de outro valor não provoca uma nova dedução de tipo.

Uma consequência importante é que `auto` exige, em uma declaração comum de variável, um inicializador a partir do qual o tipo possa ser deduzido:

```
1 auto quantidade = 100;
```

Já a construção:

```
1 auto quantidade;
```

não é válida, pois não existe um inicializador que forneça ao compilador as informações necessárias para determinar o tipo de `quantidade`.

👍 Boa Prática de Programação 1.14

Utilize `auto` quando a dedução tornar o código mais claro ou quando o tipo for excessivamente longo ou pouco relevante para a compreensão imediata da operação. Quando a explicitação do tipo contribuir para a clareza do algoritmo ou para a compreensão pedagógica do código, especificar o tipo diretamente pode ser preferível.

1.6.6 Exemplo Integrado e Comparativo

O código a seguir consolida os conceitos apresentados nesta seção, combinando declaração, inicialização por lista e atribuição.

Código 1.14: code_14.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     // Inicializacao por lista (C++11)
6     int estoqueAtual{50};
7     double precoUnitario{12.50};
8
9     std::cout << "Estoque inicial: "
10                << estoqueAtual
11                << " unidades.\n";
12
13     std::cout << "Preco por unidade: R$ "
14                << precoUnitario
15                << "\n";
16
17     // Declaracao e inicializacao de outra variavel
18     int unidadesVendidas{12};
19
20     // Atribuicao: modifica o valor de um objeto existente
21     estoqueAtual = estoqueAtual - unidadesVendidas;
22
23     std::cout << "\nApos venda de "
24                << unidadesVendidas
25                << " unidades.\n";
26
27     std::cout << "Estoque atualizado: "
28                << estoqueAtual
29                << " unidades.\n";
30     return 0;
31 }
```

As declarações:

```
1 int estoqueAtual{50};  
2 double precoUnitario{12.50};  
3 int unidadesVendidas{12};
```

definem as respectivas variáveis e estabelecem seus valores iniciais por meio de **inicialização por lista** (*list-initialization*). Posteriormente, a expressão:

```
1 estoqueAtual = estoqueAtual - unidadesVendidas;
```

realiza uma **atribuição**. Nesse momento, o objeto correspondente à variável `estoqueAtual` já existe e possui um valor. A expressão à direita do operador de atribuição é avaliada e o resultado é utilizado para atualizar o valor armazenado no objeto.

A distinção entre essas operações será importante quando forem estudados tipos definidos pelo usuário, construtores, referências e outros mecanismos da linguagem.

Boa Prática de Programação 1.15

Sempre que houver um valor inicial apropriado, inicie a variável no momento de sua definição. Essa prática torna a intenção do código explícita e reduz a possibilidade de utilização acidental de uma variável antes que seu valor tenha sido estabelecido.

Boa Prática de Programação 1.16

Em código C++ moderno, considere a inicialização por lista quando ela contribuir para a clareza e segurança do código:

```
1 double media{8.5};
```

Essa forma de inicialização fornece regras que impedem determinadas conversões implícitas potencialmente perigosas.

Erro Comum de Programação 1.14

Interpretar `auto` como uma forma de tipagem dinâmica. Em C++, o tipo de uma variável declarada com `auto` é determinado em tempo de compilação e permanece fixo durante sua existência.

⊗ Erro Comum de Programação 1.15

Declarar uma variável com `auto` sem fornecer um inicializador, como em `auto x;`. Em uma declaração comum de variável, não há informação suficiente para que o compilador deduza o tipo.

⊗ Erro Comum de Programação 1.16

Tratar o valor de uma variável local definida sem inicialização explícita como se fosse um valor residual previsível da memória. O programa deve estabelecer um valor válido antes de depender do conteúdo da variável.

1.7 Operadores Aritméticos e Expressões

A realização de cálculos numéricos e a manipulação de valores constituem operações fundamentais em praticamente todos os algoritmos. Em C++, essas operações são expressas por meio de **operadores**, que atuam sobre **operandos** e produzem resultados de acordo com as regras da linguagem.

Uma construção como `a + b` constitui uma **expressão**: seus operandos são `a` e `b`, enquanto o símbolo `+` especifica a operação de adição. O resultado da expressão depende não apenas dos valores envolvidos, mas também dos respectivos tipos.

Os operadores aritméticos básicos apresentados nesta seção são a adição (`+`), a subtração (`-`), a multiplicação (`*`), a divisão (`/`) e o módulo (`%`). Os quatro primeiros podem ser utilizados com diferentes categorias de tipos numéricos, enquanto o operador de módulo, na forma aritmética convencional apresentada neste capítulo, é utilizado com operandos inteiros.

1.7.1 Os Operadores Aritméticos Fundamentais

A linguagem C++ disponibiliza cinco operadores aritméticos binários fundamentais. Eles são denominados *binários* porque, nessa forma de utilização, cada operador atua sobre dois operandos.

A Tabela 1.3 apresenta esses operadores e alguns exemplos elementares.

Tabela 1.3: Operadores aritméticos binários fundamentais em C++.

Operador	Operação	Exemplo	Resultado
+	Adição	7 + 3	10
-	Subtração	7 - 3	4
*	Multiplicação	7 * 3	21
/	Divisão	7 / 3	2
%	Módulo	7 % 3	1

Um aspecto fundamental é que o resultado de uma operação aritmética não deve ser analisado apenas pela representação matemática da operação. Em C++, os **tipos dos operandos** participam da determinação do comportamento da expressão. Assim, embora `7 / 3` e `7.0 / 3.0` representem operações matematicamente equivalentes, as duas expressões possuem comportamentos distintos porque seus operandos pertencem a tipos numéricos diferentes.

Vamos implementar uma ideia bem simples no código a seguir:

Código 1.15: code_15.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     int resultadoInteiro = 7 / 3;
6     double resultadoReal = 7.0 / 3.0;
7
8     std::cout << "Divisao inteira: "
9               << resultadoInteiro << '\n';
10
11    std::cout << "Divisao em ponto flutuante: "
12             << resultadoReal << '\n';
13
14    return 0;
15 }
```

Nesse exemplo, `7 / 3` produz o valor inteiro 2, enquanto `7.0 / 3.0` produz um resultado

em ponto flutuante aproximadamente igual a 2.33333. Não se trata, portanto, de um arredondamento convencional de 2.33333 para 2; a primeira operação é uma **divisão inteira**.

Aviso

Em C++, o resultado de uma expressão aritmética depende dos tipos envolvidos. Por isso, expressões matematicamente semelhantes podem produzir resultados computacionais diferentes. Em particular, a divisão entre operandos inteiros produz um resultado inteiro, enquanto a presença de operandos de ponto flutuante pode produzir um resultado em ponto flutuante.

Operadores Unários + e -

Embora os cinco operadores apresentados anteriormente estejam sendo tratados aqui em sua forma **binária**, os operadores + e - também podem ser utilizados na forma **unária**, isto é, atuando sobre apenas um operando.

Por exemplo:

Código 1.16: code_16.cpp

```
1  #include <iostream>
2
3  int main()
4  {
5      int x = 10;
6
7      int positivo = +x;
8      int negativo = -x;
9
10     std::cout << "Valor positivo: " << positivo << '\n';
11     std::cout << "Valor negativo: " << negativo << '\n';
12
13     return 0;
14 }
```

Na expressão +x, o operador unário + indica o sinal positivo do operando. Na expressão

$-x$, o operador unário $-$ produz o valor correspondente à negação de x .

É importante distinguir essas formas das operações binárias apresentadas anteriormente.

Em $a - b$, por exemplo, o operador $-$ possui dois operandos e realiza uma subtração. Em

$-x$, possui apenas um operando e representa uma operação unária.

NOTA

A classificação de um operador como **unário** ou **binário** depende da forma como ele é utilizado na expressão. Portanto, o mesmo símbolo pode representar operações diferentes conforme o número de operandos sobre os quais atua.

Particularidades da Divisão Inteira e do Operador Módulo

A divisão inteira e o operador de módulo merecem atenção especial porque seu comportamento pode diferir significativamente da expectativa baseada apenas na aritmética usual. Em particular, a divisão entre valores inteiros não produz automaticamente um valor fracionário, e o operador $\%$ permite obter o restante associado à divisão inteira.

Considere, por exemplo:

Código 1.17: code_17.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     int quociente = 7 / 3;
6     int resto = 7 % 3;
7
8     std::cout << "Quociente: " << quociente << '\n';
9     std::cout << "Resto: " << resto << '\n';
10
11     return 0;
12 }
```

A expressão $7 / 3$ produz o quociente inteiro 2, enquanto $7 \% 3$ produz o valor 1, correspondente ao restante da divisão inteira.

Essas duas operações estão matematicamente relacionadas pela identidade

$$7 = 3 \times 2 + 1.$$

Assim, para os operandos inteiros utilizados nesse exemplo,

$$a = (a/b) \times b + (a\%b),$$

desde que as condições definidas pela linguagem para a operação sejam satisfeitas.

Boa Prática de Programação 1.17

Ao realizar cálculos que envolvam valores inteiros, verifique explicitamente se a divisão desejada é uma divisão inteira ou uma divisão em ponto flutuante. Uma expressão como $7 / 3$ não deve ser utilizada quando o objetivo for preservar a parte fracionária do resultado.

Erro Comum de Programação 1.17

Esperar que a expressão $7 / 3$ produza aproximadamente 2.33333. Como ambos os operandos são inteiros, a operação é realizada como divisão inteira e o resultado é 2. Para obter um resultado em ponto flutuante, pelo menos um dos operandos deve participar de uma operação aritmética em ponto flutuante, como em $7.0 / 3$.

Erro Comum de Programação 1.18

Confundir o operador de módulo (%) com uma operação que produz sempre um valor positivo. O resultado de % é determinado pelas regras da divisão inteira em C++ e seu sinal depende dos operandos. Esse comportamento será analisado em detalhe nos exemplos envolvendo valores negativos.

1.7.2 Precedência e Associatividade de Operadores

Quando uma expressão contém diferentes operadores, as regras de **precedência** e **associatividade** determinam como essa expressão é estruturada e, conseqüentemente, quais operações devem ser consideradas primeiro.

A **precedência** estabelece a prioridade relativa entre diferentes operadores. Já a **associatividade** determina como operadores que possuem a mesma precedência são agrupados quando aparecem consecutivamente em uma expressão.

É importante distinguir essas regras da **ordem de avaliação**. A precedência e a associatividade determinam principalmente a estrutura sintática da expressão; elas não devem ser interpretadas, de maneira geral, como uma garantia de que cada subexpressão será executada fisicamente da esquerda para a direita.

Para os operadores aritméticos fundamentais estudados nesta seção, podemos resumir as regras da seguinte maneira:

Tabela 1.4: Precedência e associatividade dos operadores aritméticos.

Ordem	Operadores	Operação	Associatividade
1	()	Agrupamento por parênteses	–
2	*, /, %	Multiplicação, divisão e módulo	Esquerda para direita
3	+, -	Adição e subtração	Esquerda para direita

Os parênteses possuem precedência de agrupamento e, portanto, permitem que o programador determine explicitamente a estrutura da expressão. Entre *, / e %, todos possuem a mesma precedência. O mesmo ocorre entre + e -. Quando operadores de mesma precedência aparecem em sequência, sua associatividade é da esquerda para a direita.

Regras de Agrupamento de Expressões

Considere a seguinte expressão:

```
1 int resultado = 2 + 3 * 4;
```

O operador * possui precedência maior que o operador +. Consequentemente, a expressão é estruturada como:

```
1 int resultado = 2 + (3 * 4);
```

O resultado será, portanto, 14, e não 20.

Quando desejamos alterar explicitamente esse agrupamento, podemos utilizar parênteses:

```
1 int resultado = (2 + 3) * 4;
```

Nesse caso, a adição é agrupada primeiro e o resultado é posteriormente multiplicado por 4, produzindo 20.

Operadores de Mesma Precedência

Quando operadores de mesma precedência aparecem consecutivamente, a associatividade determina seu agrupamento.

Por exemplo:

```
1 int resultado = 20 / 5 * 2;
```

Os operadores / e * possuem a mesma precedência e associatividade da esquerda para a direita. A expressão é, portanto, interpretada como:

```
1 int resultado = (20 / 5) * 2;
```

O resultado é 8.

Analogamente:

```
1 int resultado = 20 - 5 - 3;
```

é agrupada como:

```
1 int resultado = (20 - 5) - 3;
```

produzindo 12.

Uso de Parênteses para Tornar a Intenção Explícita

Embora o compilador conheça as regras de precedência e associatividade da linguagem, o uso criterioso de parênteses pode tornar a intenção matemática do programa mais evidente para o leitor.

Considere o cálculo de uma média:

```
1 double media = (nota1 + nota2 + nota3) / 3.0;
```

Os parênteses deixam explícito que as três notas devem ser somadas antes que o resultado seja dividido por 3.0.

Sem os parênteses:

```
1 double media = nota1 + nota2 + nota3 / 3.0;
```

a divisão possui precedência maior que a adição. A expressão seria interpretada como:

```
1 double media = nota1 + nota2 + (nota3 / 3.0);
```

produzindo um resultado matematicamente diferente.

1.7.3 Atribuição Composta (*Compound Assignment*)

C++ fornece operadores de atribuição composta que combinam uma operação com a atribuição do resultado à variável utilizada como operando à esquerda. Esses operadores constituem uma forma abreviada de expressões de atribuição frequentemente utilizadas em algoritmos numéricos.

Tabela 1.5: Operadores de atribuição composta.

Operador	Exemplo	Forma equivalente
+=	x += 5;	x = x + 5;
-=	x -= 3;	x = x - 3;
*=	x *= 2;	x = x * 2;
/=	x /= 4;	x = x / 4;
%=	x %= 2;	x = x % 2;

Por exemplo:

```
1 int estoque{50};
2 estoque -= 12;
```

é uma forma compacta de escrever:

```
1 int estoque{50};  
2 estoque = estoque - 12;
```

Após a operação, o valor de estoque será 38.

Os operadores de atribuição composta são especialmente úteis em estruturas de repetição e em algoritmos que atualizam progressivamente o valor de uma variável.

1.7.4 Exemplo Integrado: Decomposição de um Intervalo de Tempo

O programa a seguir demonstra a aplicação conjunta da divisão inteira, do operador módulo, da precedência e da atribuição de valores.

O objetivo é decompor uma quantidade total de segundos em horas, minutos e segundos. Uma hora possui 3600 segundos e um minuto possui 60 segundos.

Código 1.18: code_18.cpp

```
1 #include <iostream>  
2  
3 int main()  
4 {  
5     int totalSegundos{0};  
6     std::cout << "Digite o total de segundos a converter: ";  
7     std::cin >> totalSegundos;  
8  
9     // Extracao das horas por divisao inteira  
10    int horas = totalSegundos / 3600;  
11  
12    // Obtencao dos segundos restantes apos a extracao das horas  
13    int segundosRestantes = totalSegundos % 3600;  
14  
15    // Extracao dos minutos restantes  
16    int minutos = segundosRestantes / 60;  
17  
18    // Obtencao dos segundos restantes  
19    int segundos = segundosRestantes % 60;
```

```

20
21     std::cout << "\n--- Resultado da Decomposicao ---\n";
22     std::cout << totalSegundos
23         << " segundos equivalem a: "
24         << horas << "h "
25         << minutos << "m "
26         << segundos << "s\n";
27     return 0;
28 }

```

Supondo, por exemplo, que o usuário informe 7384, o programa realizará conceitualmente as seguintes operações:

- $7384 / 3600$ produz 2 horas;
- $7384 \% 3600$ produz 184 segundos restantes;
- $184 / 60$ produz 3 minutos;
- $184 \% 60$ produz 4 segundos.

Consequentemente, a saída será equivalente a:

```

1 7384 segundos equivalem a: 2h 3m 4s

```

Esse exemplo evidencia uma característica importante da divisão inteira: quando ambos os operandos são inteiros, o resultado também é determinado pelas regras da divisão inteira. O operador %, por sua vez, fornece o resto da divisão inteira.

Boa Prática de Programação 1.18

Mesmo quando as regras formais de precedência da linguagem determinam corretamente o agrupamento de uma expressão, utilize parênteses para manter a lógica mais clara. O objetivo não é “ensinar o compilador”, mas tornar o código mais legível, auditável e menos suscetível a erros de manutenção.

⊗ Erro Comum de Programação 1.19

Supor que a precedência dos operadores representa necessariamente a ordem temporal em que todas as subexpressões serão avaliadas. Precedência e associatividade determinam o agrupamento sintático da expressão; a ordem de avaliação é uma questão distinta e possui regras próprias na linguagem C++.

⊗ Erro Comum de Programação 1.20

Realizar uma divisão inteira por zero, como em $x / 0$ ou $x \% 0$. Para operandos inteiros, esse tipo de operação produz **comportamento indefinido** e não deve ser utilizada em um programa. O código deve garantir que o divisor seja diferente de zero antes da operação.

⊗ Erro Comum de Programação 1.21

Esperar um resultado decimal ao realizar uma divisão entre dois inteiros e posteriormente armazenar o resultado em uma variável do tipo `double`. Por exemplo:

```
1 double resultado = 7 / 2;
```

A divisão é realizada inicialmente entre dois operandos do tipo inteiro e produz 3. Somente depois esse valor é convertido para `double`, resultando em 3.0.

Para obter o resultado real da divisão, pelo menos um dos operandos deve participar de uma operação de ponto flutuante:

```
1 double resultado1 = 7.0 / 2;
2 double resultado2 = 7 / 2.0;
3 double resultado3 = static_cast<double>(7) / 2;
```

1.8 Desenvolvimento de Programas Simples em C++

Após o domínio dos conceitos fundamentais de estrutura básica, fluxos de Entrada/Saída, variáveis e operadores aritméticos, o próximo passo na engenharia de software em C++ envolve a consolidação dessas ferramentas no desenvolvimento de aplicações funcionais e robustas.

Nesta seção, avançaremos além da execução linear simples. Analisaremos programas que abordam a **soma de inteiros com verificação de estouro**, o **cálculo de média ponderada com conversão de tipos e validação de fluxo**, e a **comparação condicional e estatística de valores**.

1.8.1 Exemplo 1: Soma e Prevenção de Estouro de Memória (Overflow)

Ao realizar operações de soma com variáveis inteiras, existe o risco de os dados ultrapassarem o limite superior do tipo (*Arithmetic Overflow*). Em C++, o estouro de inteiros com sinal resulta em **Comportamento Indefinido** (*Undefined Behavior*).

O código a seguir realiza a leitura de dois números inteiros e valida a operação antes de efetuar a soma, utilizando limites definidos no cabeçalho `<limits>`.

Código 1.19: code_19.cpp

```
1 #include <iostream>
2 #include <limits>
3
4 int main() {
5     int a{0};
6     int b{0};
7
8     std::cout << "---- SOMA SEGURA DE DOIS INTEIROS ----\n";
9     std::cout << "Digite o primeiro numero inteiro: ";
10    std::cin >> a;
11    std::cout << "Digite o segundo numero inteiro: ";
12    std::cin >> b;
13
14    // Verificacao de estouro para soma de dois positivos
15    if (b > 0 && a > std::numeric_limits<int>::max() - b) {
16        std::cerr << "Erro: A soma resulta em estouro superior de memoria (Overflow)
17        \n";
18        return 1;
19    }
20    // Verificacao de estouro para soma de dois negativos
```

```
20     if (b < 0 && a < std::numeric_limits<int>::min() - b) {
21         std::cerr << "Erro: A soma resulta em estouro inferior de memoria (Underflow
22             )!\n";
23         return 1;
24     }
25
26     int soma = a + b;
27     std::cout << "Resultado da soma: " << soma << "\n";
28
29     return 0;
30 }
```

Em C++, o tipo `int` possui um tamanho fixo na memória (geralmente **32 bits** na maioria das arquiteturas modernas e compiladores como GCC, Clang e MSVC). Devido a essa limitação de espaço, existe um intervalo bem definido de valores que uma variável inteira com sinal (*signed int*) pode armazenar.

Representado em complemento de dois, o intervalo típico de um `int` de 32 bits é:

- **Valor Mínimo** (`std::numeric_limits<int>::min()`): $-2.147.483.648$ (-2^{31})
- **Valor Máximo** (`std::numeric_limits<int>::max()`): $+2.147.483.647$ ($2^{31} - 1$)

Quando tentamos realizar uma operação cujo resultado excede esse limite, seja para cima ou para baixo, ocorre o **estouro de memória** (*Arithmetic Overflow* ou *Underflow*).

Na linguagem C++, o estouro de inteiros com sinal resulta em **Comportamento Indefinido** (*Undefined Behavior* – UB). Isso significa que a linguagem não garante o que acontecerá após o estouro:

1. O valor pode sofrer *wrap-around* (o número “dá a volta” e se torna negativo no limite superior).
2. O compilador pode otimizar e remover o código de verificação por pressupor que um *overflow* nunca acontece.
3. O programa pode gerar resultados corrompidos ou falhar.

Para evitar esses riscos, o código apresentado realiza uma **verificação preventiva antes** de efetuar a operação $a + b$, comparando os operandos diretamente com os limites da máquina (`std::numeric_limits<int>::max()` e `min()`).

Exemplos Práticos de Entrada para Disparar a Verificação

Considerando o padrão de 32 bits, os valores exatos de entrada a seguir acionam as travas de segurança do programa:

1. Testando o Estouro Superior (*Overflow*) Para acionar a primeira validação ($b > 0$ && $a > \text{max} - b$), a soma de a e b deve resultar em um valor maior que 2.147.483.647.

- **Exemplo 1 (Limite Exato):**

- **Entrada para a:** 2147483647
- **Entrada para b:** 1
- **Análise:** $2.147.483.647 + 1 = 2.147.483.648$ (excede o limite em 1 unidade).
- **Resultado:** O programa exibirá a mensagem de erro de *Overflow*.

- **Exemplo 2 (Valores Médios Elevados):**

- **Entrada para a:** 1500000000
- **Entrada para b:** 1000000000
- **Análise:** $1.500.000.000 + 1.000.000.000 = 2.500.000.000$ (ultrapassa o limite de 2.147.483.647).
- **Resultado:** o programa bloqueará a operação antes que ocorra a falha de memória.

2. Testando o Estouro Inferior (*Underflow*) Para acionar a segunda validação ($b < 0$ && $a < \text{min} - b$), a soma de dois números negativos deve resultar em um valor menor (mais negativo) que $-2.147.483.648$.

- **Exemplo 1 (Limite Exato Negativo):**

- **Entrada para a:** -2147483648
- **Entrada para b:** -1
- **Análise:** $-2.147.483.648 + (-1) = -2.147.483.649$ (ultrapassa o limite inferior em 1 unidade).
- **Resultado:** O programa exibirá a mensagem de erro de *Underflow*.

• **Exemplo 2 (Soma de Negativos Grandes):**

- **Entrada para a:** -1200000000
- **Entrada para b:** -1000000000
- **Análise:** $-1.200.000.000 + (-1.000.000.000) = -2.200.000.000$ (excede o limite de -2.147.483.648).
- **Resultado:** O programa bloqueará a execução de forma segura.

Tabela 1.6: Casos de teste para verificação de estouro de memória.

Teste	Entrada a	Entrada b	Condição Ativada	Saída Esperada
Operação Válida	2000000000	1000000000	Nenhuma (Soma OK)	Resultado da soma: 2100000000
<i>Overflow</i> (Superior)	2147483600	100	$b > 0 \ \&\& \ a > \text{max} - b$	Erro: A soma resulta em estouro superior...
<i>Underflow</i> (Inferior)	-2000000000	-2000000000	$b < 0 \ \&\& \ a < \text{min} - b$	Erro: A soma resulta em estouro inferior...

1.8.2 Exemplo 2: Processamento Estatístico de Médias e Tratamento do Fluxo de Entrada

Um erro comum em programas de console ocorre quando o usuário digita letras ou caracteres inválidos ao aguardar a leitura de um número. Isso coloca o fluxo `std::cin` em um estado de falha (*fail state*), fazendo com que leituras subsequentes falhem silenciosamente.

O exemplo a seguir calcula a **média aritmética ponderada** de um estudante, realizando a conversão explícita de tipos com `static_cast` e garantindo a **higienização do buffer de entrada**.

Código 1.20: code_20.cpp

```
1  #include <iostream>
2  #include <limits>
3  #include <iomanip> // Necessario para std::setprecision e std::fixed
4
5  // Funcao auxiliar para realizar a leitura segura de um double positivo
6  double lerNotaSegura(const std::string& prompt) {
7      double valor{0.0};
8      while (true) {
9          std::cout << prompt;
10         if (std::cin >> valor && valor >= 0.0 && valor <= 10.0) {
11             return valor; // Leitura bem-sucedida e valor dentro da faixa valida
12         }
13         // Trata a falha de leitura
14         std::cout << "Entrada invalida! Digite uma nota numerica entre 0.0 e 10.0.\n"
15             << "\n";
16         std::cin.clear(); // Limpa as sinalizacoes de erro (failbit) do std::cin
17         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); //
18             // Descarta entrada invalida
19     }
20 }
21
22 int main() {
23     std::cout << "--- CALCULO DE MEDIA PONDERADA ---\n";
24     double nota1 = lerNotaSegura("Digite a Nota 1 (Peso 2): ");
25     double nota2 = lerNotaSegura("Digite a Nota 2 (Peso 3): ");
26     double nota3 = lerNotaSegura("Digite a Nota 3 (Peso 5): ");
27     constexpr int peso1{2};
28     constexpr int peso2{3};
29     constexpr int peso3{5};
30
31     // Conversao explicita de inteiros para double usando static_cast
32     double somaPesos = static_cast<double>(peso1 + peso2 + peso3);
33     double mediaPonderada = ((nota1 * peso1) + (nota2 * peso2) + (nota3 * peso3)) /
```

```

    somaPesos;
32 // Formatacao da saida com 2 casas decimais
33 std::cout << std::fixed << std::setprecision(2);
34 std::cout << "\n--- RELATORIO FINAL ---\n";
35 std::cout << "Media Ponderada Apurada: " << mediaPonderada << "\n";
36
37 if (mediaPonderada >= 7.0) {
38     std::cout << "Status: APROVADO\n";
39 } else if (mediaPonderada >= 5.0) {
40     std::cout << "Status: EM RECUPERACAO\n";
41 } else {
42     std::cout << "Status: REPROVADO\n";
43 }
44 return 0;
45 }

```

O processamento seguro de dados via console exige atenção a dois fatores críticos: a **recuperação de falhas de leitura no `std::cin`** e a **precisão na conversão explícita de tipos**.

Quando o **compilador** tenta extrair dados para uma variável do tipo `double` utilizando `std::cin` » `valor` e encontra caracteres **não** numéricos (como letras ou símbolos), ocorre que:

1. A **flag de erro `failbit`** do **fluxo `std::cin`** é ativada;
2. A variável `valor` **não** recebe o dado incorreto;
3. O texto inválido **permanece** intocado no *buffer* de entrada padrão.

Se nenhuma ação for tomada, as chamadas subsequentes de leitura tentarão reprocessar o mesmo texto inválido presas no *buffer*, gerando um *loop* infinito de erros. A função `lerNotaSegura` contorna essa situação em duas etapas:

- `std::cin.clear()`: **redefine** o estado interno do `std::cin`, **removendo** a flag de erro e permitindo novas operações de I/O.

- `std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');`
descarta todos os caracteres pendentes no *buffer* até encontrar o caractere de nova linha (`'\n'`), garantindo a higienização completa para a próxima tentativa.

No processo de conversão explícita com `static_cast` os pesos são declarados como constantes inteiras (`constexpr int`). Para evitar a coerção implícita durante a **divisão aritmética** e explicitar o rigor matemático do algoritmo, utiliza-se a **conversão explícita** em tempo de compilação:

```
double somaPesos = static_cast<double>(peso1 + peso2 + peso3);
```

Garante-se que a divisão subsequente ocorra **integralmente** no domínio de ponto flutuante com **precisão dupla**.

A seguir, apresentamos simulações de execução com valores numéricos para demonstrar o comportamento do algoritmo diante de entradas válidas e falhas de digitação.

1. Para o cenário com Entrada Inválida e Recuperação de Erro, considere a seguinte sequência digitada pelo usuário no terminal:

- **Nota 1:** o usuário digita `abc` e pressiona Enter.
 - *Ação:* a leitura falha, o programa exibe a mensagem de aviso, executa `clear()` e `ignore()`, eliminando `abc` do *buffer*.
- **Nota 1 (Nova tentativa):** o usuário digita `8.5`.
 - *Ação:* entrada válida dentro do intervalo `[0.0, 10.0]`. Valor atribuído à `nota1`.
- **Nota 2:** o usuário digita `15.0` (fora da faixa estipulada).
 - *Ação:* a validação `valor <= 10.0` falha. O programa rejeita a nota e solicita novamente.
- **Nota 2 (Nova tentativa):** o usuário digita `6.0`.
- **Nota 3:** O usuário digita `4.0`.

2. Para o cálculo da Média Ponderada para o exemplo, com as entradas aceitas ($\text{Nota}_1 = 8.5$, $\text{Nota}_2 = 6.0$, $\text{Nota}_3 = 4.0$) e os pesos definidos ($w_1 = 2$, $w_2 = 3$, $w_3 = 5$):

$$\text{Média} = \frac{(8.5 \times 2) + (6.0 \times 3) + (4.0 \times 5)}{2 + 3 + 5} = \frac{17.0 + 18.0 + 20.0}{10.0} = \frac{55.0}{10.0} = 5.50$$

Resultado impresso: O relatório final indicará a média 5.50 e o status EM RECUPERACAO.

Tabela 1.7: Casos de teste para cálculo de média ponderada e validação de fluxo.

Cenário	Entradas Digitadas	Notas Válidas Aceitas	Média Apu-rada	Resultado / Status
Aprovação Di-reta	8.0, 7.5, 9.0	8.0, 7.5, 9.0	8.35	Status: APROVADO
Entrada com Letras	erro, 6.0, 12.0, 5.0, 4.0	6.0, 5.0, 4.0	4.70	Status: REPROVADO (com 2 rejeições)
Limiares Míni-mos	5.0, 5.0, 5.0	5.0, 5.0, 5.0	5.00	Status: EM RECUPERACAO
Limiares Máxi-mos	10.0, 10.0, 10.0	10.0, 10.0, 10.0	10.00	Status: APROVADO

1.8.3 Exemplo 3: Análise Comparativa Multi-Variável e Retorno via `std::optional`

O terceiro programa explora a **comparação** e **extração estatística** sobre um conjunto de valores. Para lidar com situações em que o resultado da análise pode ser **indefinido** (por exemplo, buscar a mediana ou amplitude em um conjunto de dados vazio), o C++17 introduziu o **tipo utilitário `std::optional<T>`**.

O código abaixo lê três números distintos, determina o **maior**, o **menor**, o **valor intermediário** (*mediana*) e verifica se há **equivalência estrutural**.

Código 1.21: code_21.cpp

```

1 #include <iostream>
2 #include <algorithm> // Necessario para std::max e std::min
3 #include <optional>

```

```
4
5 struct AnaliseValores {
6     double menor;
7     double intermediario;
8     double maior;
9     bool todosIguais;
10 };
11
12 // Analisa e ordena tres valores numericos
13 AnaliseValores analisarTresValores(double a, double b, double c) {
14     if (a == b && b == c) {
15         return {a, b, c, true};
16     }
17
18     double menor = std::min({a, b, c});
19     double maior = std::max({a, b, c});
20     // O valor intermediario e obtido calculando a soma total e subtraindo os
21     // extremos
22     double intermediario = (a + b + c) - (menor + maior);
23
24     return {menor, intermediario, maior, false};
25 }
26
27 int main() {
28     double x{0.0}, y{0.0}, z{0.0};
29     std::cout << "---- ANALISADOR COMPARATIVO DE TRES VALORES ----\n";
30     std::cout << "Digite tres numeros reais separados por espaco: ";
31
32     if (!(std::cin >> x >> y >> z)) {
33         std::cerr << "Erro: Entrada de dados invalida.\n";
34         return 1;
35     }
36     AnaliseValores resultado = analisarTresValores(x, y, z);
37     std::cout << "\n--- RESULTADO DA ANALISE ---\n";
38     if (resultado.todosIguais) {
39         std::cout << "Os tres valores fornecidos sao rigorosamente iguais: " <<
40             resultado.menor << "\n";
41     } else {
42         std::cout << "Menor valor: " << resultado.menor << "\n";
43     }
44 }
```

```

41     std::cout << "Valor intermediario: " << resultado.intermediario << "\n";
42     std::cout << "Maior valor:          " << resultado.maior << "\n";
43     std::cout << "Amplitude (Maior - Menor): " << (resultado.maior - resultado.
        menor) << "\n";
44 }
45 return 0;
46 }

```

No contexto da lógica do programa de análise comparativa, a **equivalência estrutural** ocorre quando todos os componentes do conjunto de dados possuem exatamente o mesmo valor e tipo, tornando-os indistinguíveis sob o ponto de vista da ordenação.

Matematicamente, para um conjunto $S = \{a, b, c\}$, a equivalência estrutural é satisfeita se, e somente se:

$$a = b = c$$

Quando essa condição é satisfeita, conceitos como *maior*, *menor* ou *amplitude* perdem a relevância prática, visto que a amplitude é nula ($Maior - Menor = 0$).

Por outro lado, quando **não há equivalência estrutural** ($a \neq b \vee b \neq c$), o algoritmo realiza a partição do conjunto, determinando rigorosamente:

1. O **limite inferior** ($\min(a, b, c)$);
2. O **limite superior** ($\max(a, b, c)$);
3. O **elemento intermediário** ou mediana local, obtido por propriedade algébrica:

$$Intermediário = (a + b + c) - (\min(a, b, c) + \max(a, b, c))$$

4. A **amplitude estatística** ($\max - \min$), que quantifica a dispersão dos dados.

Exemplos Práticos de Entrada para Casos sem Equivalência

Abaixo são apresentados cenários práticos de execução ilustrando como o programa se comporta quando as entradas não são estruturalmente equivalentes.

1. Valores Totalmente Distintos em Ordem Desordenada

- **Entrada:** 15.5 4.2 9.8
- **Análise:**
 - $a = 15.5, b = 4.2, c = 9.8$.
 - Verificação de igualdade: $15.5 \neq 4.2 \implies$ **Sem equivalência estrutural.**
 - Menor: 4.2 | Intermediário: 9.8 | Maior: 15.5.
 - Amplitude: $15.5 - 4.2 = 11.3$.

2. Equivalência Parcial (Dois Valores Iguais)

- **Entrada:** 7.0 12.0 7.0
- **Análise:**
 - $a = 7.0, b = 12.0, c = 7.0$.
 - Verificação de igualdade: $a == c$, mas $a \neq b \implies$ **Sem equivalência estrutural global.**
 - Menor: 7.0 | Intermediário: 7.0 | Maior: 12.0.
 - Amplitude: $12.0 - 7.0 = 5.0$.

3. Entradas Negativas e Ponto Flutuante

- **Entrada:** -10.5 -2.0 -5.5
- **Análise:**
 - $a = -10.5, b = -2.0, c = -5.5$.
 - Verificação de igualdade: $a \neq b \neq c \implies$ **Sem equivalência estrutural.**
 - Menor: -10.5 | Intermediário: -5.5 | Maior: -2.0.
 - Amplitude: $-2.0 - (-10.5) = 8.5$.

Tabela 1.8: Casos de teste para verificação de equivalência estrutural e ordenamento.

Cenário	Entrada	Equivalência?	Saída Produzida pelo Programa
Equivalência Total	5.0 5.0 5.0	Sim ($a = b = c$)	Os tres valores fornecidos sao rigorosamente iguais: 5
Sem Equivalência (Geral)	15.5 4.2 9.8	Não ($a \neq b \neq c$)	Menor: 4.2 Intermediario: 9.8 Maior: 15.5 Amplitude: 11.3
Equivalência Parcial	7.0 12.0 7.0	Não ($a = c \neq b$)	Menor: 7 Intermediario: 7 Maior: 12 Amplitude: 5
Valores Negativos	-10.5 -2.0 -5.5	Não ($a \neq b \neq c$)	Menor: -10.5 Intermediario: -5.5 Maior: -2 Amplitude: 8.5

👍 Boa Prática de Programação 1.19

Sempre valide o estado de `std::cin` imediatamente após tentar ler dados fornecidos pelo usuário. Tratar possíveis erros de digitação e limpar o *stream* via `std::cin.clear()` e `std::cin.ignore()` evita que seu aplicativo entre em *loops* infinitos de falha.

👍 Boa Prática de Programação 1.20

Utilize a conversão explícita `static_cast<tipo>(expressao)` em vez do estilo antigo de coerção herdado do C `((tipo)expressao)`. O `static_cast` é verificado em tempo de compilação e explicita com clareza a intenção da conversão de tipo no código.

⚠ Erro Comum de Programação 1.22

Comparar valores em ponto flutuante (`float` ou `double`) utilizando o operador de igualdade exata (`==`) em cálculos complexos. Devido a imprecisão na representação binária de números fracionários, prefira comparar se a diferença absoluta entre eles é menor que uma margem de tolerância (*epsilon*): `std::abs(a - b) < 0.00001`.

1.9 Revisão Conceitual

Esta seção reúne distinções terminológicas e conceitos fundamentais introduzidos ao longo do capítulo. Seu objetivo é servir como material de revisão e como referência rápida para evitar ambiguidades na terminologia utilizada ao longo do texto.

A tabela a seguir reúne alguns dos principais termos utilizados ao longo deste capítulo, apresentando suas definições, exemplos e observações importantes para sua correta interpretação.

Tabela 1.9: Principais elementos e conceitos da linguagem C++.

Termo	Definição	Exemplo em C++	Observação
Programa	Conjunto de elementos de código-fonte que, segundo as regras da linguagem, descreve um comportamento computacional.	Um arquivo ou conjunto de arquivos que implementa uma aplicação.	Não confundir programa com uma única função ou instrução.
Função	Unidade de código que pode ser invocada para executar uma determinada operação e, opcionalmente, produzir um valor de retorno.	<code>int main()</code>	Uma função possui uma assinatura, um corpo e pode receber parâmetros e retornar um valor.
Instrução (statement)	Construção sintática que representa uma ação ou controle de execução dentro de um programa.	<code>return 0;</code>	Nem todo elemento de um programa é uma instrução. Declarações e expressões podem ter papéis sintáticos distintos.
Expressão (expression)	Construção sintática que representa um cálculo ou operação e, em geral, possui um resultado determinado por suas regras semânticas.	<code>a + b</code>	Uma expressão pode aparecer dentro de uma instrução, como em <code>int c = a + b;</code> .
Continua na próxima página			

Continuação da Tabela 1.9			
Termo	Definição	Exemplo em C++	Observação
Declaração (declaration)	Construção que introduz ou declara uma entidade no programa, tornando seu nome e suas propriedades conhecidos em determinado contexto.	<code>int idade;</code>	Uma declaração pode ou não também constituir uma definição, dependendo da entidade e da construção utilizada.
Definição (definition)	Construção que fornece a definição de uma entidade ou, quando aplicável, sua implementação completa.	<code>int idade = 20;</code>	O conceito é mais específico que o de declaração: uma definição é uma declaração que fornece a informação necessária para definir a entidade.
Diretiva de pré-processamento	Diretiva destinada ao pré-processador, que realiza transformações ou outras operações antes da compilação propriamente dita.	<code>#include</code>	Não é uma instrução C++ convencional e não termina necessariamente com ponto e vírgula.
Identificador	Nome utilizado para identificar entidades introduzidas no programa.	<code>idade, contador</code> <code>main,</code>	Identificadores seguem regras específicas da linguagem e podem representar diferentes tipos de entidades.
Palavra-chave (keyword)	Palavra reservada pela linguagem para uma finalidade sintática ou semântica específica.	<code>int, return, class, if</code>	Não pode ser utilizada livremente como identificador.
Literal	Representação textual de um valor constante no código-fonte.	<code>10, 3.14, 'A', "Olá"</code>	O tipo e a interpretação dependem da forma como o literal é escrito.
Variável	Objeto que possui uma região de armazenamento e pode ter seu valor alterado durante a execução, respeitadas as regras de seu tipo e de seu contexto.	<code>int contador = 0;</code>	“Variável” não é sinônimo de “identificador”: o identificador é o nome; a variável é a entidade associada.
Continua na próxima página			

Continuação da Tabela 1.9			
Termo	Definição	Exemplo em C++	Observação
Tipo	Entidade da linguagem que determina propriedades relevantes de valores e objetos, incluindo sua representação e as operações permitidas.	int, double, char, bool	O sistema de tipos é fundamental para a interpretação das expressões e operações.
Objeto	Região de armazenamento que contém um valor e possui um tipo associado durante sua existência.	int contador = 10;	Uma variável pode designar um objeto, mas os conceitos de “variável” e “objeto” não são completamente equivalentes.
Operador	Símbolo ou construção que especifica uma operação aplicada a um ou mais operandos.	+, -, =, ==	Alguns operadores são sobrecarregáveis e podem apresentar comportamentos diferentes conforme os tipos dos operandos.
Operandos	Expressões ou entidades sobre as quais um operador atua.	Em a + b, a e b são operandos.	A quantidade de operandos depende do operador.
Comentário	Texto inserido no código-fonte para documentação e compreensão humana, ignorado pelo compilador como parte da semântica do programa.	// Calcula a média	Comentários não alteram diretamente o comportamento do programa.
Bloco	Sequência de declarações e instruções delimitada por chaves.	Um conjunto de instruções entre { e }	Blocos também estabelecem escopos em diversas situações.
Escopo (scope)	Região do programa na qual uma determinada entidade pode ser referenciada por seu nome de acordo com as regras da linguagem.	O escopo de uma variável local declarada dentro de uma função.	Escopo é uma propriedade do código-fonte; não deve ser confundido com tempo de vida.

Apêndice A: Projeto Prático - Módulo 1: Modelando o Cão Robótico (IRD)

1. Contexto e Objetivo do Projeto

Bem-vindo à equipe de desenvolvimento da **PEts That Think** (PETT)! Neste primeiro módulo do projeto prático, nosso objetivo é projetar e implementar o módulo base do **Incredible Robot Dog (IRD)**.

2. Mapeamento Conceitual: Da Metáfora ao C++

Para organizar o projeto, utilizaremos a divisão de papéis introduzida no capítulo:

- **A Engenharia (Objetos, Atributos e Métodos):** mapeamento das partes do robô (como *Eyes* e *Mouth*) com suas características físicas/estado (atributos) e suas ações (métodos ou funções).
- **A Reatividade (Programação Orientada a Eventos):** associação de estímulos externos (ex: presença de comida) a uma alteração de estado interna e uma resposta comportamental (ex: babar ou latir).
- **A Armazenagem (Variáveis e Tipos):** uso da memória como uma "folha de papel anotada" contendo o tipo da informação, o nome do identificador e seu valor.

Tabela 1.10: Mapeamento dos conceitos para a Orientação a Objetos em C++.

Conceito	Termo Técnico C++	Exemplo no Projeto IRD
Objeto / Componente	Classe / Instância (<i>class</i>)	<i>Mouth</i> , <i>Eyes</i> , <i>RobotDog</i>
Atributo	Variável de Membro / Estado	<i>salivaLevel</i> , <i>energySupply</i>
Método / Habilidade	Função / Método de Membro	<i>eat()</i> , <i>bark()</i> , <i>see()</i>
Evento	Interrupção / Gatilho de Entrada	Inserção de alimento / Toque na tela

3. Especificação dos Exercícios do Apêndice

Os alunos deverão implementar o esqueleto inicial do robô em C++ seguindo as três etapas abaixo:

Etapa 1: Declaração de Variáveis e Tipos (O “Bloco de Notas” do IRD)

Com base na explicação da memória do computador como uma tabela tipo/nome/valor, declare as variáveis necessárias para controlar o status do IRD:

- `int height` (altura em cm)
- `int weight` (peso em kg)
- `float energysupply` (nível de energia em %)
- `bool isHungry` (indicador booleano de fome)

Etapa 2: Implementação da Estrutura de Objetos e Métodos

Crie a representação simplificada da classe `RobotDog` que agrupa os métodos de rotina diária (como `wake()`, `eat()`, `sleep()`) e os atributos do sistema.

Etapa 3: Simulação de Eventos (Causa e Efeito)

Implemente uma rotina que simule a reação a um evento externo: quando a função `detectFood()` é acionada, o nível de saliva aumenta para 100% e o método de latido/saliva é disparado.

4. Código de Referência do Módulo 1

Abaixo encontra-se a implementação modelo do projeto para validação dos alunos:

Código 1.22: `code_22.cpp`

```

1 #include <iostream>
2 #include <string>
3
4 // Representacao do objeto Boca do IRD (Atributos e Metodos)
5 class Mouth {
6 private:
7     int teethCount{32};           // Atributo: numero de dentes
8     double salivaLevel{0.0};      // Atributo: nivel de saliva (0.0 a 100.0)
9
10 public:
11     // Metodo que executa uma acao e altera estado interno
12     void eat(const std::string& food) {
13         std::cout << "[Boca] Mastigando e comendo: " << food << "\n";
14         salivaLevel = 10.0; // Reseta o nivel de saliva apos comer
15     }
16
17     // Metodo acionado por evento externo
18     void triggerDrool() {
19         salivaLevel = 100.0;
20         std::cout << "[Boca] Alimento detectado! Nivel de saliva em "
21             << salivaLevel << "%. (Drooling...)\n";
22     }
23 };
24
25 // Objeto Principal: IRD (Incredible Robot Dog)
26 class RobotDog {
27 private:
28     // Atributos de estado do Robo (Tabela de Memoria)
29     int height{30};               // cm
30     int weight{12};               // kg
31     double energySupply{100.0};   // porcentagem (float/double)
32     Mouth mouth;                  // Componente/Objeto interno
33
34 public:
35     // Metodos da Rotina Diaria (Daily Routine)
36     void wake() {
37         std::cout << "[IRD] Acordando e ligando sistemas...\n";
38     }

```

```
39
40 void sleep() {
41     std::cout << "[IRD] Entrando em modo de repouso (Sleep)...\n";
42 }
43
44 // Simulacao de Evento: Visao de Comida (Pavlov Event)
45 void seeFood(const std::string& food) {
46     std::cout << "\n--- EVENTO: Visao de Comida (" << food << ") ---\n";
47     mouth.triggerDrool(); // Causa e Efeito
48     mouth.eat(food);
49     energySupply = std::min(100.0, energySupply + 15.0);
50 }
51
52 void displayStatus() const {
53     std::cout << "\n[STATUS IRD] Energia: " << energySupply
54         << "% | Peso: " << weight << "kg\n";
55 }
56 };
57
58 int main() {
59     std::cout << "=== PROJETO PETT: INICIALIZANDO IRD ===\n\n";
60
61     RobotDog myDog;
62
63     // Execucao sequencial do programa (Program Flow)
64     myDog.wake();
65     myDog.displayStatus();
66
67     // Disparo de Evento Externo
68     myDog.seeFood("Leftover Pizza");
69
70     myDog.displayStatus();
71     myDog.sleep();
72
73     return 0;
74 }
```


5. Guia de Acompanhamento e Entregáveis do Aluno

Para a avaliação deste módulo do apêndice, o aluno deverá entregar:

1. Diagrama conceitual mapeando ao menos **2 novos objetos** para o cão (ex: Tail, Ears) com pelo menos **2 atributos** e **1 método** cada.
2. Implementação do programa em C++ compilável contendo a sequência da rotina diária customizada (adicionando métodos como `walk()` e `play()`).

Referências Bibliográficas

- [1] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. *Programming languages — C++*. Geneva: International Organization for Standardization, 2024.
- [2] DEITEL, P.; DEITEL, H. M. *C++ How to Program*. 10. ed. Boston: Pearson, 2017. ISBN 9780134448237.
- [3] cppreference.com. *Main function*. 2026. Disponível em: <https://en.cppreference.com/w/cpp/language/main_function>.
- [4] STROUSTRUP, B. *Programming: Principles and Practice Using C++*. 3. ed. Boston: Addison-Wesley Professional, 2024. ISBN 978-0-13-830868-1.
- [5] cppreference.com. *Return statement*. 2026. Disponível em: <<https://en.cppreference.com/w/cpp/language/return>>.
- [6] cppreference.com. *Standard library header <iostream>*. 2026. Disponível em: <<https://en.cppreference.com/w/cpp/header/iostream>>.
- [7] cppreference.com. *Namespaces*. 2026. Disponível em: <<https://en.cppreference.com/w/cpp/language/namespace>>.
- [8] DEITEL, P.; DEITEL, H. *C++ How to Program*. 10. ed. Harlow: Pearson Education Limited, 2020. ISBN 9781292153414.
- [9] cppreference.com. *std::cout, std::wcout*. 2026. Documentação de referência da biblioteca padrão de C++. Disponível em: <<https://en.cppreference.com/cpp/io/cout>>.
- [10] cppreference.com. *std::cin, std::wcin*. 2026. Documentação de referência da biblioteca padrão de C++. Disponível em: <<https://en.cppreference.com/cpp/io/cin>>.

- [11] cppreference.com. *std::basic_ostream*. 2026. Documentação de referência da biblioteca padrão de C++. Disponível em: <https://en.cppreference.com/cpp/io/basic_ostream>.
- [12] cppreference.com. *Operators*. 2026. Documentação de referência da linguagem C++. Disponível em: <<https://en.cppreference.com/w/cpp/language/operators>>.
- [13] cppreference.com. *std::endl*. 2026. Documentação de referência da biblioteca padrão de C++. Disponível em: <<https://en.cppreference.com/cpp/io/manip/endl>>.
- [14] cppreference.com. *std::flush*. 2026. Documentação de referência da biblioteca padrão de C++. Disponível em: <<https://en.cppreference.com/w/cpp/io/manip/flush>>.
- [15] cppreference.com. *std::basic_istream*. 2026. Documentação de referência da biblioteca padrão de C++. Disponível em: <https://en.cppreference.com/w/cpp/io/basic_istream>.
- [16] cppreference.com. *std::getline*. 2026. Documentação de referência da biblioteca padrão de C++. Disponível em: <https://en.cppreference.com/cpp/string/basic_string/getline>.
- [17] cppreference.com. *std::format*. 2026. Documentação de referência da linguagem e da biblioteca padrão de C++. Disponível em: <<https://en.cppreference.com/w/cpp/utility/format/format>>.